

Research papers

Grounding large language models in hydrologic modelling

Yingjia Li^a, Shiruo Hu^a, Feng Zhang^b, Xinpeng Yu^a, Wei Luo^{a,b}, Dingxiao Liu^c,
Bin Xu^d, Jianshi Zhao^{a,*}

^a State Key Laboratory of Hydro-Science and Engineering, Department of Hydraulic Engineering, Tsinghua University, Beijing 10084, China

^b CHN Energy Dadu River Big Data Services Co., Ltd., Sichuan 610041, China

^c Zhipu AI, Beijing 100084, China

^d Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China

ARTICLE INFO

This manuscript was handled by Dan Lu, Editor-in-Chief, with the assistance of Phong Le, Associate Editor

Keywords:

Multi-agent architecture

Time series forecasting

Automated machine learning

Agentic workflow

ABSTRACT

Although large language models (LLMs) possess extensive hydrological knowledge, they often struggle to appropriately simulate complex physical processes in hydrological systems. To address this limitation, we present an LLM-based agentic intelligent modelling approach for hydrological time-series forecasting (HydroAIM). HydroAIM decouples the multi-agent architecture through the model context protocol (MCP) to integrate an expert task workflow, a standardised algorithmic workflow (SAW) library, and an iterative feedback closed-loop execution toolbox. Comprehensive experiments demonstrated that across 125 modelling tasks of varying types utilising five LLMs, HydroAIM achieved an execution success rate of 92.8%. Ablation studies have revealed that removing the expert task workflow causes complete system failure owing to planning chaos, plummeting the success rate to 0%. This result indicates that long-horizon task planning is the core bottleneck restricting LLMs from executing hydrological modelling tasks. Furthermore, when applied to 531 catchments in the Catchment Attributes and Meteorology for Large-sample Studies (CAMELS) dataset without human intervention, HydroAIM achieves median Nash-Sutcliffe efficiency (NSE) values of 0.58 and 0.73 for local and global modelling, respectively, achieving usable modelling performance. This study demonstrates that, when properly supported and constrained, LLM-based agents can be effectively grounded in automated hydrological modelling, providing a scalable pathway for quantitative tasks in specialised scientific domains.

1. Introduction

Large language models (LLMs) have demonstrated exceptional capabilities for semantic understanding, cross-task generalisation, and code generation (Boiko et al., 2023; Wang et al., 2023). These advancements offer unprecedented opportunities to explore LLM-based techniques in specialised scientific domains, thereby drawing considerable attention from the hydrologic research community (Kadiyala et al., 2024; Kojima et al., 2022). Previous studies have demonstrated the remarkable applications of LLMs in answering water-related queries, remote sensing image analysis (Ren et al., 2024), hydrologic literature mining (Miao et al., 2024), fundamental reasoning (Kizilkaya et al., 2025), and early flood warnings (Martelo et al., 2026). Notably, LLMs still face major challenges in these tasks owing to gaps in domain knowledge and response stability (Kizilkaya et al., 2025).

Despite their ability to realise these knowledge-based tasks, the use of LLMs for quantitative modelling is more challenging. Recently,

scholars have made preliminary attempts to apply LLMs to hydrological modelling (Sun et al., 2025). Data-driven deep-learning models are generally employed as targets, such as deep learning for time-series models in streamflow forecasting (Kratzert et al., 2018; Kratzert et al., 2019), estimation of evapotranspiration (Wu et al., 2019), and water-level prediction (Ma et al., 2023). These deep-learning models perfectly align with the core strengths of LLMs in terms of code generation and logical reasoning. However, directly applying general LLMs to complex hydrological modelling tasks encounters three inherent and critical challenges. First, the absence of physical and algorithmic boundary constraints renders them highly susceptible to model architecture and hyperparameter hallucinations (Alansari and Luqman, 2025; Ji et al., 2023). Second, when confronted with ultra-long workflows that encompass data preprocessing, model construction, and training tuning, they are prone to context degradation and planning disorientation caused by information overload (Liu, 2024b; Huang and Zhang, 2025). Finally, the lack of an authentic computational environment and

* Corresponding author.

E-mail address: zhaojianshi@tsinghua.edu.cn (J. Zhao).

<https://doi.org/10.1016/j.jhydrol.2026.136358>

Received 11 May 2026; Received in revised form 11 July 2026; Accepted 2 September 2026

Available online 6 September 2026

0022-1694/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

closed-loop feedback mechanism often renders the generated code inexecutable (Yang et al., 2024). Thus, an approach that integrates the capacities of LLMs and professional modelling skills is required to address the aforementioned challenges.

The LLM-based multi-agent architecture offers a potential pathway to ground LLMs in hydrological modelling (Maldonado et al., 2024). By using LLM-based orchestrating agents to decompose complex tasks, coordinate specialised agents, and invoke external tools (Dong et al., 2024; Shen et al., 2023b), such architectures have automated specialised workflows in fields such as medicine and materials science (Feng et al., 2025; Lei et al., 2024). More recently, agentic AI has also received increasing attention in the water sector. Conceptual multi-agent frameworks have been proposed for water engineering and urban water management, where an orchestrating agent coordinates specialised agents and external tools to support monitoring, planning, simulation, and decision-making (Fu, 2026; Goldshtein et al., 2025; Hosseini et al., 2026; Yang and Chiou, 2026). Related implementations have connected LLM agents with established hydraulic and urban drainage simulators, such as EPANET, SWMM, and physics-based groundwater models, enabling natural-language-driven model configuration, simulation execution, result analysis, and provenance tracking (Ma et al., 2026; Wang et al., 2026; Zhang and Valeo, 2026). In hydrology, AI-augmented modelling frameworks have further explored the use of specialised LLM agents to assist with model conceptualisation, configuration, execution, interpretation, and calibration (Eythorsson and Clark, 2025; Tudaji et al., 2026; Zhu et al., 2026). These studies demonstrate the potential of agentic AI to lower the barrier to complex water and environmental modelling workflows. However, most existing water-sector agentic systems either remain at the conceptual decision-support level or focus on orchestrating established process-based simulators. Relatively little attention has been devoted to investigating whether LLM agents can autonomously construct, train, debug, and evaluate data-driven hydrological time-series models under standardised modelling constraints. Hydrological deep-learning modelling involves data preprocessing, architecture design, hyperparameter configuration, training diagnosis, and quantitative evaluation, all of which require domain-specific algorithmic skills beyond the general planning capabilities of LLMs. Therefore, a methodological gap remains between general LLM-based multi-agent architectures and their reliable application to end-to-end hydrological deep-learning modelling.

The LLM-based multi-agent architecture offers a potential pathway to ground LLMs in hydrological modelling (Maldonado et al., 2024). By employing LLMs as ‘central controllers’ that autonomously decompose tasks and orchestrate external tools (Dong et al., 2024; Shen et al., 2023b), this architecture has successfully automated specialised workflows in interdisciplinary fields, such as medicine (Feng et al., 2025), materials science (Lei et al., 2024), and traditional water distribution systems (Goldshtein et al., 2025). These cross-domain successes provide crucial insights for applying LLMs to hydrological modelling. However, hydrological modelling involves data preprocessing, architectural design, and training tuning, all of which rely on professional domain-specific skills beyond the capabilities of general LLMs. Notably, knowledge and methodological gaps still exist between the general LLM-based multi-agent architecture and its application in hydrological modelling.

To fill this gap, in this study, we propose an LLM-based agentic intelligent modelling approach for hydrological time-series forecasting (HydroAIM). HydroAIM introduces three core mechanisms at an architectural level. First, to address the hallucinations that are prone to occur during unconstrained code generation, a standardised algorithmic workflow (SAW) library is introduced to provide the LLM with expert-verified algorithmic references. Second, to address the context degradation and weak planning capabilities in long-horizon scientific workflows, a model context protocol (MCP)-based multi-agent architecture guided by expert task workflows is implemented. By presetting standardised domain-specific task chains, this architecture logically decouples complex modelling tasks and prevents system failures caused by

blind trial and error. Third, to overcome the lack of authentic execution environments and computational verification, HydroAIM is equipped with a customised toolbox and closed-loop execution feedback engine. This engine bridges the physical gap between tool invocation and code execution, providing an agent with the capacity for autonomous trial-and-error and quantitative iterations.

Furthermore, in this study, we aim to explore effective pathways to enhance the transferability of the proposed approach, specifically focusing on the following three questions: (1) can general LLMs facilitated by multi-agent architectures and constrained generation mechanisms overcome inherent hallucinations and logical disconnections to autonomously complete end-to-end quantitative hydrologic modelling from task analysis to model training? (2) Within the automated modelling closed loop, what architectural components play a decisive role in mitigating the limitations of LLMs and ensuring the task success rate and simulation accuracy? (3) When tackling complex hydrologic time-series forecasting tasks, can data-driven workflows autonomously orchestrated by agents achieve application benchmarks established by human experts or traditional physics-based models without human intervention?

The remainder of this study is organised as follows. Section 2 introduces HydroAIM. Section 3 outlines the experimental design and datasets. Section 4 presents the results, including agentic execution capability, hydrological modelling performance, and capability expansion. Section 5 discusses the limitations, reproducibility considerations, and practical implications of autonomous LLM-based hydrological modelling. Finally, Section 6 presents the conclusions.

2. Methodology

2.1. Overview of HydroAIM

The proposed HydroAIM employs LLMs as the central controller, integrating an MCP-based multi-agent collaboration framework (Ray, 2025), SAW library, and closed-loop execution engine. The core concept is to split the complex, long-horizon deep-learning modelling workflow into a series of logically decoupled subtasks that are collaboratively handled by agents with distinct roles, thereby enabling an automated end-to-end process from task breakdown to result generation (Dong et al., 2024). The overall structure of HydroAIM is illustrated in Fig. 1.

In HydroAIM, the LLM serves as a central controller with three primary functions: task planning, dynamic tool invocation, and result integration. To logically decouple the complex hydrologic modelling process, HydroAIM adopts a multi-agent architecture guided by a structured expert agentic workflow. In particular, the approach defines four specialised agents: a hydrologic task-analysis agent responsible for understanding problem constraints and defining modelling objectives; data preprocessing agent for data cleaning, outlier detection, and feature engineering; model construction agent handling architecture selection, hyperparameter configuration, and code generation; and result-presentation agent managing the training loop, performance evaluation, and visualisation of prediction curves.

To enable effective collaboration, the MCP serves as a central communication hub. Through this standardised protocol, the four agents can asynchronously read and write states without direct coupling, while also maintaining a unified interface between the LLM and external physical environment.

A workspace (W) is constructed to support the physical execution of the agents. The workspace represents the execution environment of the method and is responsible for storing task objectives, datasets, code, and runtime feedback, thereby ensuring traceability and reusability across different stages (Fatouros et al., 2025). Accordingly, HydroAIM defines a set of four specialised agents, namely, $A = \{a_1, a_2, a_3, a_4\}$. In particular, the hydrologic task-analysis, data preprocessing, model-construction, and result-presentation agents were used. During execution, the

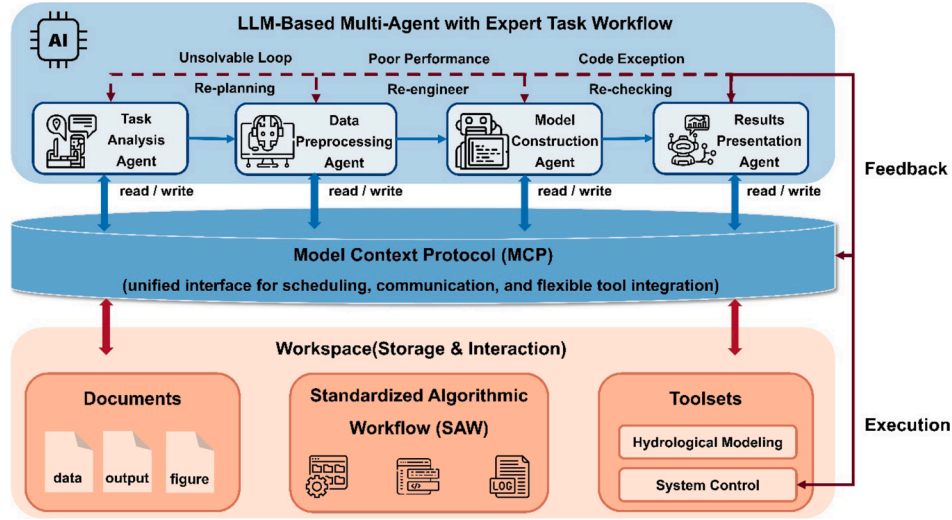


Fig. 1. Overall workflow of the large language model (LLM)-based agentic intelligent modelling approach for hydrologic time-series forecasting (HydroAIM). The approach features a decoupled, multi-agent design guided by an expert task workflow to ensure a standardised modelling progression. The model context protocol (MCP) acts as a centralised interface for scheduling and communication, enabling agents to asynchronously read and write states without direct coupling. In the physical execution workspace, agents autonomously invoke the standardised algorithmic workflow (SAW) library and domain-specific toolsets to execute rigorous quantitative tasks. Crucially, HydroAIM incorporates an adaptive closed-loop feedback mechanism, and upon encountering specific exceptions, the execution engine autonomously routes the error traceback to the corresponding agent for targeted refinement.

method leverages MCP to coordinate tool invocation and iteratively achieves code generation, debugging, and execution. The overall process can be expressed as follows:

$$S_{i+1} = F(S_i, T, W), i = 0, 1, \dots, n \quad (1)$$

$$S_i = (D_i, C_i, M_i, R_i) \quad (2)$$

where S_i represents the comprehensive system state at the i -th iteration, comprising the data state (D_i), generated code files (C_i), model definitions (M_i), and runtime feedback (R_i); T represents the task requirements; W represents the workspace; and $F(\cdot)$ is the adaptive task-update mechanism driven by the agent set A .

Notably, the state transition $F(\cdot)$ is not a rigid uniform update but an adaptive routing mechanism triggered by the feedback R_i . Depending on the error type, HydroAIM autonomously triggers the targeted refinements: re-checking the code (C_{i+1}) for code exceptions, re-engineering data features (D_{i+1}) for poor predictive performance, or in rare unsolvable loops, re-planning the fundamental architecture (M_{i+1}). Through this conditional trial-and-error process, HydroAIM progressively refines the scheme to meet the task objectives.

In the experimental setting of this study, HydroAIM was designed and evaluated as an autonomous execution framework rather than a step-by-step human-in-the-loop system. Human researchers provide the initial modelling objective, candidate datasets, available toolsets, the SAW library, and evaluation criteria before execution. Once a run is initiated, the specialised agents autonomously complete task analysis, data preprocessing, model construction, code execution, debugging, and result generation without manual correction of intermediate outputs. Therefore, human involvement in the reported experiments is limited to pre-run task and environment specification and post-run result assessment. This setting allows us to evaluate the autonomous modelling capability of LLM-based agents under controlled and reproducible conditions.

In the implementation, all LLM backbones were accessed via provider APIs rather than locally deployed. Each specialised agent was instantiated via a role-specific system prompt and received the current task state, relevant workspace files, tool specifications, and selected SAW references as context. To reduce ambiguity in downstream execution, agent outputs were constrained to complete executable code

blocks, structured tool-call arguments, or concise state-update reports. Information exchange among agents was mediated by the MCP-managed workspace rather than free-form dialogue. Specifically, datasets, generated scripts, model definitions, runtime logs, and feedback reports were stored as shared states in the workspace and asynchronously read or updated by different agents during the modelling process.

2.2. Model context protocol (MCP)

The MCP serves as the architectural backbone of the proposed HydroAIM, fundamentally enabling the decoupling of the probabilistic reasoning of the LLM from the deterministic execution of hydrological modelling tools (Anthropic, 2024b). In contrast to traditional hard-coded workflows, the MCP establishes a standardised communication layer that enables the LLM to interact with external computational environments in a structured and safe manner.

Structurally, HydroAIM adopts the host-client-server topology of MCP. The LLM-based host layer functions as the reasoning and orchestration center. It interprets high-level hydrological forecasting objectives, decomposes them into atomic executable tasks, and determines which tools should be invoked according to the current modelling context. The MCP client, embedded within the host layer, is responsible for protocol-level communication with MCP servers, including establishing connections, formatting tool-call requests, transmitting parameters, and receiving execution results. Conversely, the MCP servers act as execution engines, encapsulating domain-specific functionalities ranging from time-series data preprocessing and tensor construction to deep-learning model training and metric evaluation. The servers expose these functionalities through standardised JSON-based schemas, which define tool names, required input parameters, data types, and return formats. These schemas form a contract between the generative LLM-based host and deterministic modelling tools.

The dynamic interaction within this architecture follows a reasoning-action-observation closed loop, as illustrated in Fig. 2. The process is initiated by the LLM-based host generating a modelling plan and dispatching structured invocation requests to the appropriate MCP servers through the MCP client. Upon receiving the request, the server executes the corresponding Python-based operations and returns the execution feedback, which includes the numerical results, runtime logs, and error

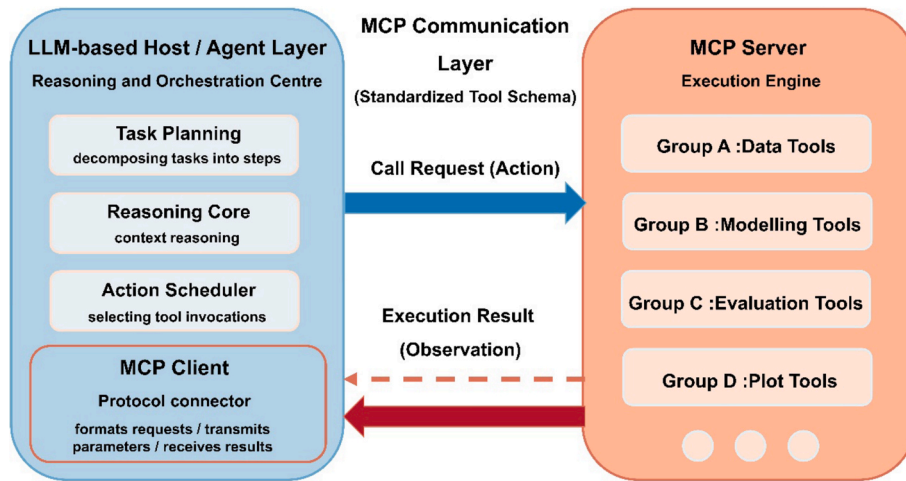


Fig. 2. Architectural interaction diagram of the model context protocol (MCP). The left panel represents the LLM-based host/agent layer, which performs reasoning, task planning, and tool-invocation selection. The MCP client is embedded within this host layer as a protocol connector responsible for formatting tool-call requests, transmitting parameters, and receiving results. The right panel represents the MCP server layer, which encapsulates hydrological tools for data pre-processing, model training, evaluation, and visualisation. The interaction follows a structured reasoning-action-observation loop: the host dispatches standardised tool-call requests through the MCP client, and the server returns computational results and runtime logs as observations, enabling dynamic workflow adjustment.

stacks. This feedback mechanism is critical as it transforms the system from a linear execution pipeline into an adaptive agent. By analysing the “observation” returned by the server, the LLM-based host can dynamically revise its strategy, such as switching from model training to hyperparameter tuning or attempting an alternative data imputation method, thereby achieving a flexible, self-correcting modelling workflow.

2.3. Workspace

In HydroAIM, the workspace serves as the central operational environment for multi-agent collaboration. It not only functions as a static storage repository but also as a dynamic state hub that facilitates the seamless exchange of task objectives, heterogeneous datasets, executable tools, and runtime feedback (Zhang et al., 2025a). To support the complete cycle of automated hydrological modelling, the workspace is

officially organised into three functional dimensions: document management, handling raw data, generated outputs, and execution feedback; toolset integration, managing the specialised hydrologic and system utilities; and the SAW library, providing structured modelling template references. These components collectively ensure the reproducibility and traceability of the entire modelling process.

2.3.1. Document management

In HydroAIM, the workspace functions as a centralised governance layer for task definitions and document assets. These documents include raw hydrological data, intermediate computational outputs, and generated analytical figures.

For input-data management, we adopted a metadata-driven architecture to ensure consistent access (Fang et al., 2021). As illustrated in Fig. 3, using the Catchment Attributes and Meteorology for Large-sample Studies (CAMELS) dataset as an example, the workspace

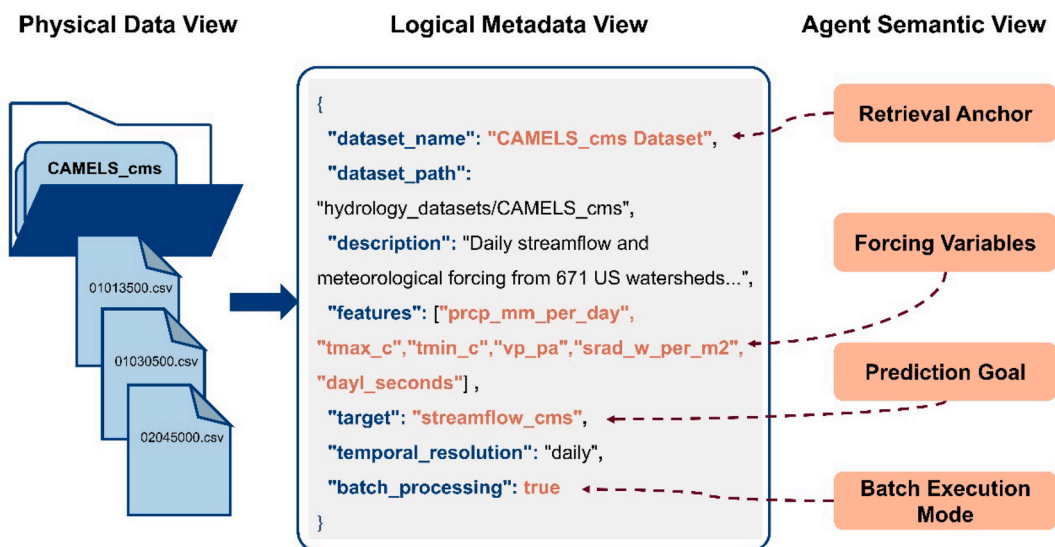


Fig. 3. Metadata-driven document management in the HydroAIM workspace. HydroAIM organises data through a three-tier abstraction: (Left) the physical data view stores raw hydrologic assets (e.g. Catchment Attributes and Meteorology for Large-sample Studies (CAMELS) basin files). (Center) The logical metadata view indexes these assets through a unified JavaScript Object Notation (JSON) registry, strictly defining standardised properties ranging from unit specifications (e.g. streamflow_cms) to workflow control flags (e.g. batch_processing). (Right) The agent semantic view interprets these attributes into cognitive actionable contexts, mapping technical fields to semantic concepts, such as ‘forcing variables’ and ‘batch execution mode’.

organises data through a three-tier abstraction. In particular, raw files in the physical data view are indexed and standardised by the logical metadata view through a JSON registry, which the agent semantic view ultimately translates into actionable cognitive concepts (Addor et al., 2017; Newman et al., 2015; Tarboton et al., 2014). This three-tiered structure provides a machine-readable interface for autonomous tool invocation. For instance, when the task-analysis agent receives a request for ‘rainfall-runoff modelling across US basins’, it leverages the agent semantic view to identify the CAMELS dataset. Upon detecting the “batch_processing: true” flag, the agents automatically activate a multi-basin iterator to sequentially load data, achieving an automated workflow without manual intervention.

Considering dynamic assets, the workspace archives both the final modelling deliverables (such as predictive curves and analytical figures) and intermediate runtime states. The latter were converted and saved as structured logs. To ensure scientific rigor, this module serves as a provenance tracker for the entire modelling lifecycle. It records all executed Python codes, diagnostic feedback (such as error stacks and performance metrics), and execution logs. These logs act as the “observations” in the agent’s reasoning loop, providing the necessary evidence for autonomous debugging. Ultimately, detailed tracking transforms the reasoning process into a transparent workflow, helping scholars identify errors and ensure complete reproducibility.

2.3.2. Toolset integration

To support the completely autonomous workflow of HydroAIM, the workspace integrates a toolset implemented in Python and exposed through the MCP. The toolset was organised into two primary functional categories to balance domain specificity with system flexibility.

The first category is dedicated to the core hydrological modelling workflow. Modules, such as `data_cleaning`, `target_normalizer`, and `model_utils`, encapsulate the specialised algorithms required for time-series analysis. These tools enable the agent to efficiently handle data preprocessing tasks, such as missing value imputation and variable normalisation, and construct deep-learning models, such as long short-term memory (LSTM) and transformer, directly from the workspace. Furthermore, the `metrics_calculator` and `experiment_logger` modules help standardise the evaluation process by automatically computing key performance indicators and tracking training convergence.

The second category encompasses systems and cognitive support.

Modules, including `shell_tools`, `file_server`, and `web_search`, equip an agent with the ability to perceive and interact with its computing environment. This design enables the agent to autonomously manage workspace directories, monitor system resources, and retrieve external parameter references from the web. Complementary tools, such as calculator and code analysis, further assist in mathematical verification and code debugging, ensuring that the approach remains robust and adaptable throughout the modelling lifecycle. Simultaneously, this modularity and extensibility enables HydroAIM to meet diverse hydrological modelling requirements. Decoupling tool implementation from agent reasoning establishes a streamlined pathway for the future integration of novel data sources and their corresponding processing tools with minimal friction (Kohl et al., 2024). The executable tools integrated into HydroAIM and their corresponding functional roles are summarised in Table 1.

2.3.3. Standardised algorithmic workflow (SAW) library

In HydroAIM, the design of the SAW library aims to balance the generative limitations of LLMs with the rigorous requirements of hydrological modelling. Numerous benchmark experiments have demonstrated that although LLMs exhibit task decomposition capabilities, their ability to independently accomplish end-to-end machine-learning workflows remains limited, particularly in scenarios involving complex model architectures and long-term training processes (Chan et al., 2024; Huang et al., 2023; Starace et al., 2025). Nevertheless, in hydrology, LLMs have shown remarkable potential in modelling tasks centred on time-series prediction (Zhang et al., 2025b). To bridge this gap, we introduce a SAW library that encapsulates verified code skeletons, enabling agents to invoke robust workflows while avoiding the instability of scratchpad code generation.

As illustrated in the modular architecture shown in Fig. 4, the SAW library is organised into four sequential domain-specific modules to support a complete deep-learning lifecycle. The data loader module provides architecture-aligned data-ingestion pipelines. It includes specific templates, such as `encoder_decoder` for sequence-to-sequence architectures, `encoder_only` for non-autoregressive models, and `simple_input` for basic regression networks. These templates were automatically selected to match the input tensor requirements of the selected model architecture. The tensor-processing module ensures consistent scaling across the training and inference phases. Notably, the

Table 1

List of executable tools integrated into the large language model (LLM)-based agentic intelligent modelling approach for hydrologic time-series forecasting (HydroAIM), categorised by their functional role.

Category	Module	Key Tools	Function Description
Hydrologic Modelling	<code>data_cleaning</code>	<code>fill_missing_values</code> , <code>detect_outliers</code> , <code>split_train_test</code> , ...	Handles time-series imputation, outlier removal, and dataset partitioning
	<code>experiment_logger</code>	<code>record_metrics</code> , <code>export_results</code> , ...	Tracks loss curves and validates convergence
	<code>metrics_calculator</code>	<code>calculate_hydrology_metrics</code> , <code>export_results</code> , ...	Computes standardised hydrologic performance indicators (e.g. NSE, RMSE, etc.)
	<code>model_utils</code>	<code>load_template</code> , <code>train_model</code> , <code>evaluate_model</code> , ...	Constructs, trains, and evaluates DL models
	<code>plotting</code>	<code>line_chart</code> , <code>heatmap</code> , <code>multi_panel_plot</code> , ...	Generates visual comparisons of observed vs. predicted hydrographs
	<code>target_normalizer</code>	<code>normalize_target_variable</code> , <code>inverse_transform_predictions</code> , ...	Manages reversible scaling of hydrologic targets
System Control	<code>calculator</code>	<code>calculate</code> , <code>convert_units</code> , <code>solve_equation</code> , ...	Conducts auxiliary mathematical verification and unit conversion
	<code>code_analysis</code>	<code>analyze_code_structure</code> , <code>traverse_dirs</code> , ...	Enables the agent to comprehend and debug existing codebases
	<code>file_server</code>	<code>read/write_file</code> , <code>list_files</code> , <code>copy/delete_files</code> , ...	Manages workspace artifacts and directory structures
	<code>python_runner</code>	<code>run_python_script</code> , <code>run_python_inline</code> , ...	Provides a sandbox for executing generated modelling codes
	<code>shell_tools</code>	<code>run_command</code> , <code>check_disk_space</code> , <code>list_process</code> , ...	Monitors system resources and executes OS-level commands
	<code>web_search</code>	<code>web_search</code>	Retrieves external domain knowledge or parameter references

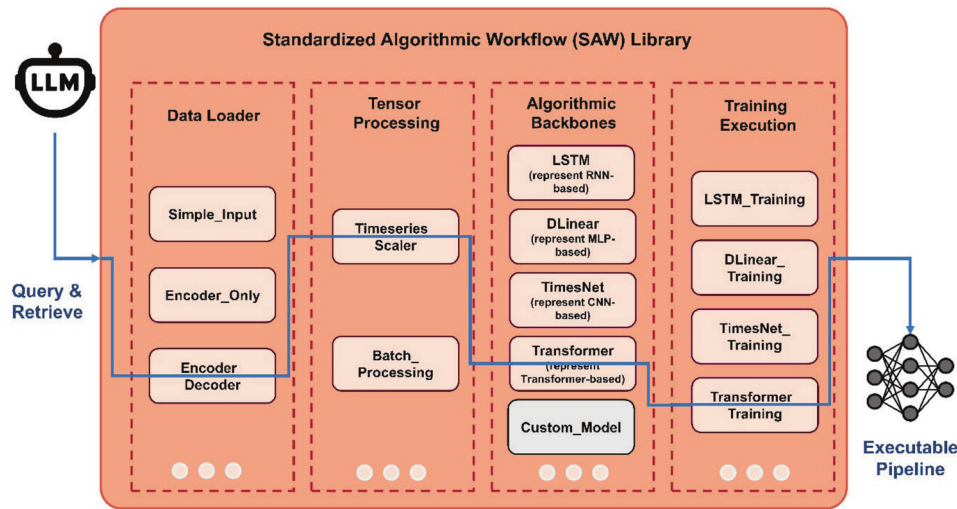


Fig. 4. Modular architecture of the standardised algorithmic workflow (SAW) library in HydroAIM. The library is structured into four sequential modules encompassing the modelling lifecycle: data loader, tensor processing, algorithmic backbones, and training execution. The blue line illustrates a concrete example of large language model (LLM)-driven dynamic assembly, where an agent queries and seamlessly connects specific templates across these modules to construct a tailored executable pipeline. Notably, the ‘Custom_Model’ slot highlights the approach’s extensibility, enabling scholars to insert novel algorithms into the standardised workflow. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

algorithmic backbone module stores a comprehensive collection of deep-learning architectures spanning diverse paradigms. This repository features recurrent neural network (RNN)-based models, such as the classic LSTM (Hochreiter and Schmidhuber, 1997), convolutional neural network (CNN)-based models, such as TimesNet and modern temporal convolutional network (ModernTCN) (Luo and Wang, 2024; Wu et al., 2023), and multilayer perceptron (MLP)-based models, such as DLinear (Zeng et al., 2023). Furthermore, it incorporates a wide array of transformer-based models, ranging from the original transformer to

state-of-the-art variants, including Autoformer, Crossformer, patch time-series transformer (PatchTST), and iTransformer (Wu et al., 2021; Zhang and Yan, 2023; Liu, et al., 2024c; Nie et al., 2023; Vaswani, et al., 2017). These are paired with the training execution module, which encapsulates model-specific optimisation loops to ensure stability. During execution, the agent dynamically queries and seamlessly connects specific templates across the four modules to construct a tailored executable pipeline. Beyond these pre-integrated algorithms, the library is designed as an open-ended framework; thus, scholars can seamlessly

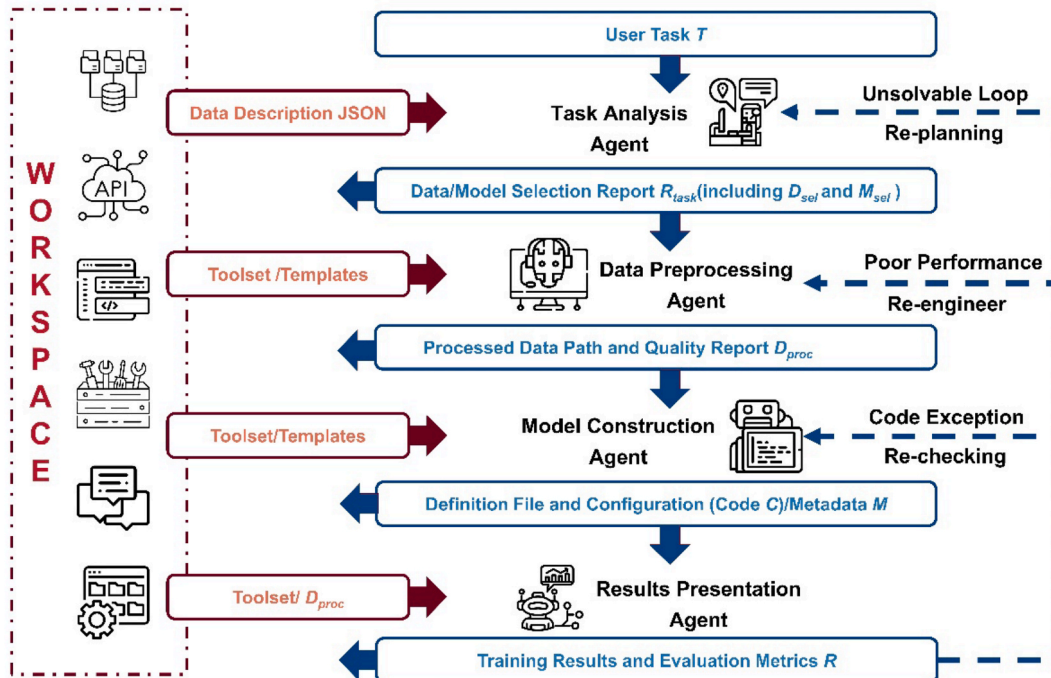


Fig. 5. Iterative multi-agent collaboration mechanism in HydroAIM. The workflow orchestrates four specialised agents, namely, task analysis, data pre-processing, model construction, and result presentation, in a sequential loop to transform user tasks into executable models. Each execution step interacts with the centralised workspace (W) to exchange standardised assets, including data description JavaScript Object Notations (JSONs), toolsets, templates, and generated code or metadata. The final training results and evaluation metrics feed back into the initial analysis stage, forming a closed-loop mechanism for continuous iterative refinement.

incorporate additional custom architectures by strictly adhering to a standardised template format.

Functionally, the SAW library complements the toolset (Section 2.3.2) through a clear division of labour. Although general-purpose functions (e.g. data cleaning) are handled by tools, the modelling stage is characterised by high structural variability. For instance, CNNs significantly differ from transformers in both their forward propagation and optimisation strategies. By encapsulating these complex task-dependent pipelines as verified templates within the SAW library, the multi-agent architecture can rapidly and safely assemble architectures tailored to specific hydrological requirements. This decoupling not only effectively mitigates the inherent coding hallucinations of LLMs but also ensures the continuous extensibility and adaptive capacity of the modelling framework in the long term (Kratzert et al., 2022; Zhmoginov et al., 2021).

2.4. Multi-agent collaboration

In this section, we formalise the multi-agent collaboration workflow within HydroAIM. By defining clear responsibility boundaries through system prompts, four specialised agents iteratively cooperate to transform abstract user requirements into executable deep-learning models. As illustrated in Fig. 5, this process relies on standardised data transmission pathways provided by the workspace.

To drive dynamic evolution, the approach collaboratively manages the global state $S_i = (D_i, C_i, M_i, R_i)$ through the MCP. Within this shared context, D_i and M_i serve as the structural targets. C_i represents the executable scripts co-developed by agents across various stages, and R_i captures the runtime feedback. The collaboration proceeds through four sequential state transitions.

Step 1: Intent parsing and data selection (a_1). The task analysis agent (a_1) interprets the user task T and global workspace context W . It identifies the prediction objectives and constraints to select the optimal subset from the candidate datasets and updates the state with the selected data object D_i^{sel} :

$$a_1 : (T, W, S_i) \mapsto S'_i = S_i \cup \{D_i^{sel}\} \quad (3)$$

Step 2: Data standardisation (a_2). Upon receiving the selection report, the data preprocessing agent (a_2) invokes the toolset to conduct cleaning, normalisation, and partitioning. This operation transforms the raw selection D_i^{sel} into a standardised, model-ready dataset D_i^{proc} , updating the data state as follows:

$$a_2 : (D_i^{sel}, toolsets) \mapsto D_i^{proc}, S'_i = (S'_i \setminus D_i^{sel}) \cup \{D_i^{proc}\} \quad (4)$$

Step 3: Model construction (a_3). Based on the processed data and SAW library, the model construction agent (a_3) designs the network architecture and generates the training pipeline. This step inserts the executable code C_{i+1} and model definition M_{i+1} into HydroAIM.

$$a_3 : (D_i^{proc}, templates) \mapsto (C_{i+1}, M_{i+1}), S''_i = S'_i \cup \{C_{i+1}, M_{i+1}\} \quad (5)$$

Step 4: Execution and validation (a_4). Finally, the resulting presentation agent (a_4) executes the training scripts. It produces comprehensive performance metrics and visualisation curves encapsulated as the feedback result in R_{i+1} :

$$a_4 : (D_i^{proc}, M_{i+1}) \mapsto R_{i+1}, S_{i+1} = S''_i \cup \{R_{i+1}\} \quad (6)$$

Through this mechanism, the output of each stage strictly serves as the input for the subsequent stage within a unified communication framework. The entire iterative optimisation process can be mathematically expressed as a composition of agent actions:

$$S_{i+1} = (a_4 \circ a_3 \circ a_2 \circ a_1)(S_i, T, W), i = 0, 1, 2... \quad (7)$$

This cyclic pipeline, spanning requirement analysis, data preparation,

model construction, and result validation, forms a closed-loop feedback mechanism (Maldonado et al., 2024; Yu et al., 2025). The feedback R_{i+1} from the current iteration provides evidence for the subsequent cycle, enabling agents to adaptively refine the model structure or preprocessing strategy, thereby addressing the inherent complexity of hydrological modelling tasks. During this autonomous collaboration process, intermediate outputs from specialised agents are not manually inspected or corrected by humans. Instead, they are verified through executable constraints and quantitative feedback within the workspace. For example, preprocessing outputs must be readable by the subsequent model-construction scripts, generated code must pass runtime execution, and model outputs must produce valid evaluation metrics and visualisation results. When these checks fail, the runtime feedback is written back to the shared state and triggers targeted revision by the corresponding agent. This design emphasises machine-verifiable reliability during autonomous execution, while leaving expert judgement on hydrological plausibility and practical acceptability to the final assessment stage.

3. Experimental designs

To evaluate the performance of HydroAIM, we devised a comprehensive experimental design encompassing two complementary dimensions: evaluating the (1) autonomous workflow execution capability and robustness of the approach under complex engineering constraints, and (2) hydrologic forecasting performance against standardised scientific benchmarks. To comprehensively evaluate the proposed HydroAIM approach, we utilised two distinct datasets. The specific data sources, variables, temporal resolutions, and corresponding experimental objectives are listed in

3.1. Phase 1: Approach capability evaluation

3.1.1. Operational dataset

As presented in Table 2, the operational dataset used for the evaluation of the capability of the approach comprises actual monitoring data derived from the scheduling operations of several cascade hydropower stations in the Dadu River Basin. Located in Southwest China as a major tributary of the Yangtze River, this basin has complex landforms and a dense network of hydropower and hydrological stations (Fig. 6). The close connections and operational rules between the upstream and downstream hydropower stations in this network considerably hinder

Table 2

Summary of dataset characteristics and corresponding experimental designs.

Dataset	Operational dataset	CAMELS dataset
Data sources	Several cascade hydropower stations in the Dadu River Basin (e.g. Houziyan, Pubugou)	531 standardised catchments across the contiguous United States (US)
Variables	Multi-source monitoring data, including the water level (m), inflow (m^3/s), power generation load (MW), and others	Extended Maurer meteorological forcings and 27 static catchment physical attributes
Temporal resolution	Multi-scale (e.g. 5-min, 1-h, 1-day), depending on the actual monitoring statistics	Standardised daily scale
Availability	Proprietary (not open-sourced)	Open-sourced
Corresponding experiment Characteristics and reasons	Phase 1: approach capability evaluation Completely simulates actual engineering conditions (including noise and unaligned scales) to rigorously verify the feasibility and robustness of the proposed approach	Phase 2: hydrologic performance evaluation Provides open benchmark data and published model outputs, enabling reproducible evaluation against established CAMELS hydrological and deep-learning benchmarks

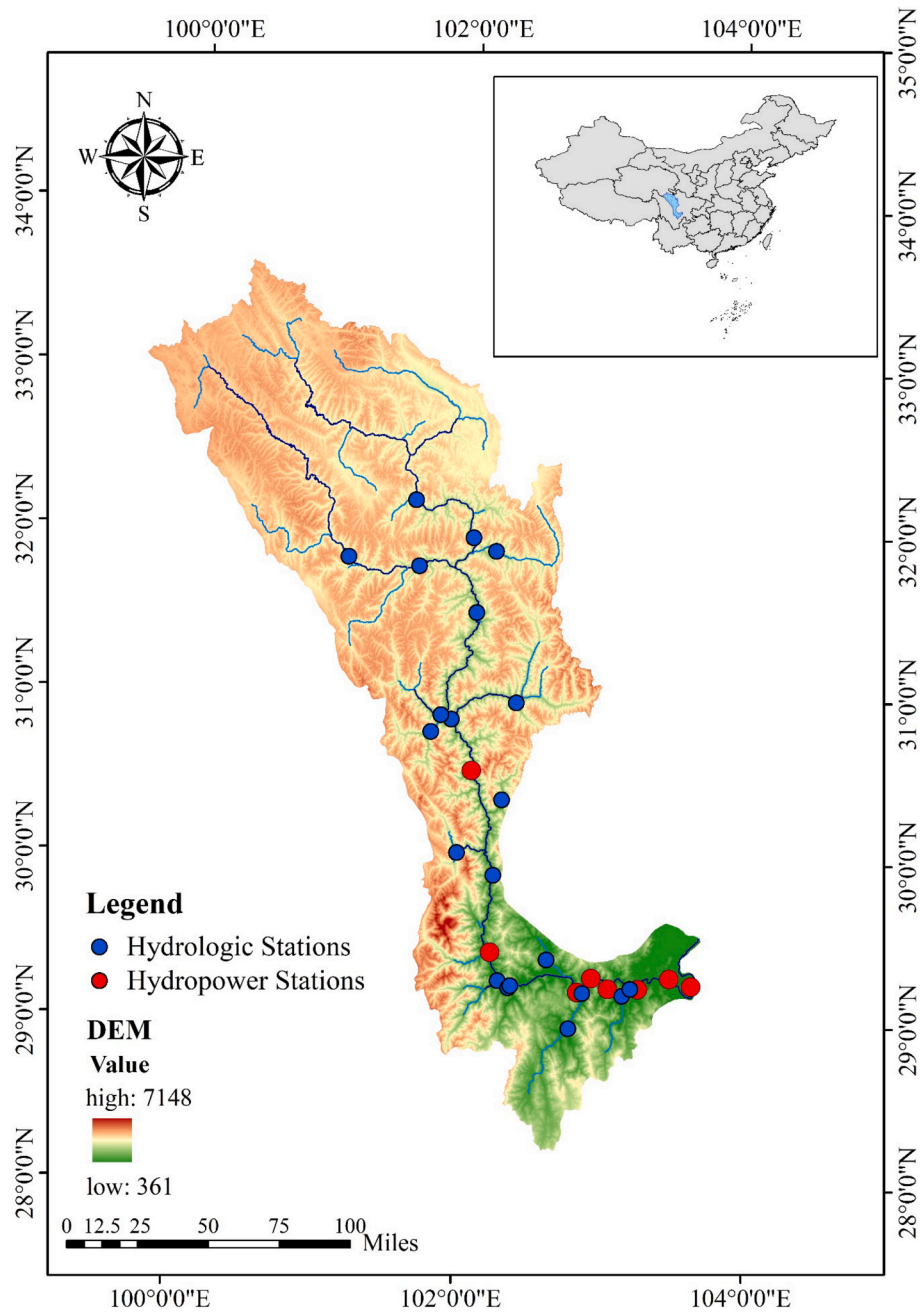


Fig. 6. Overview of the Dadu River Basin in Southwest China. This basin serves as the geographic source for the operational dataset used in evaluating the capability of the proposed approach. The map details the spatial distribution of cascade hydropower stations and hydrologic stations over a digital elevation model (DEM) background.

reservoir scheduling, and depend heavily on accurate real-time streamflow forecasting. Furthermore, these monitoring records are large, contain numerous types of variables, and have different timescales that do not match. Consequently, this environment serves as a strict and typical engineering testbed for evaluating the robustness of the HydroAIM approach when dealing with messy raw data and executing modelling tasks under real-world conditions.

3.1.2. Task configurations

In this study, five experimental groups were established to systematically evaluate the performance of HydroAIM across two dimensions: single-instance adaptability and batch-processing stability, as listed in Table 3.

The first four groups constituted a 2×2 controlled experimental

Table 3

Configurations of the hydrologic modelling tasks.

Task group	Target scope	Input features	Model architecture
Group I	Single basin/station	Multivariate	Pre-specified
Group II	Single basin/station	Univariate	Pre-specified
Group III	Single basin/station	Multivariate	Autonomous selection
Group IV	Single basin/station	Univariate	Autonomous selection
Group V	Batch scale (multi-basin/station)	Diverse	Autonomous selection

matrix focused on single-basin (or single-station) tasks. This matrix

assesses the refined modelling capability and generalisability of the approach across two factors: the complexity of the input features and autonomy of the model architecture. Building upon these single-instance tests, a fifth experimental group was introduced to validate the potential of the approach for large-scale hydrologic operations: batch automated modelling. In contrast to the first four groups, this task required the agent to autonomously generate and execute generalised batch-processing scripts. In particular, the workflow must retrieve multiple datasets from designated directories and iteratively execute pre-processing, training, and evaluation. This setup rigorously evaluated the robustness of the approach in constructing complex loop logic and managing long-sequence tasks without human intervention.

3.1.3. Comparative and ablation settings

The experiment comprised two parts, namely, baseline experiments and ablation studies, aimed at evaluating HydroAIM from both model-driven and architectural perspectives.

To explicitly address the first core scientific question, namely, whether general-purpose LLMs can overcome inherent hallucinations to autonomously complete end-to-end quantitative modelling under strict mechanistic constraints, we designed a multi-model adaptability evaluation. In particular, five representative LLMs were integrated: DeepSeek (Guo, et al., 2025; Liu, et al., 2024a), GPT (Achiam et al., 2023; OpenAI, 2025), Qwen (Yang et al., 2025), Claude (Anthropic, 2024), and Gemini (Gemini and Google, 2025). This experimental setup focused on verifying the adaptability of the approach when driven by different underlying kernels, ensuring reliable end-to-end modelling workflows regardless of the specific LLM employed.

To explicitly address the second core scientific question, namely, to identify the decisive architectural elements that determine the execution success rate of automated modelling, we designed a comprehensive ablation study. In particular, we established four ablation variants along with the full HydroAIM to isolate and quantify the necessity of each core module. DeepSeek-v3.2 as selected as the foundation for these experiments owing to its exceptional balance between reasoning capability and computational efficiency, making it suitable for extensive iterative testing. As presented in Table 4, we established four ablation variants alongside the full HydroAIM to strictly evaluate the contribution of each core module. W/o multi-agent (architecture ablation): replaces the multi-agent team with a monolithic single agent to verify the necessity of task decoupling; w/o SAW library (constraint ablation): removes the external SAW library, forcing the agents to generate specific deep-learning network structures and hyperparameter configurations based solely on internal parametric knowledge, thus lacking external algorithmic boundaries; w/o feedback (mechanism ablation): disables the closed-loop self-correction mechanism to test the robustness of the approach; and w/o workflow (workflow ablation): removes the complete expert task workflow, enabling free-form execution. We empirically validated the effectiveness of each component by systematically analysing the performance variations across these configurations.

3.1.4. Evaluation metric

We selected the task completion success rate as the primary metric for evaluating robustness. Each experimental configuration was

independently executed five times. The experiment adopted a strict end-to-end full-process completion standard as the criterion for success. In particular, a run was counted as a complete success only when it autonomously completed the entire modelling workflow without human intervention, generated executable code without unresolved syntax or runtime errors, produced a complete experimental report with valid hydrological evaluation metrics, and achieved a final NSE of at least 0.50. This threshold is consistent with widely used watershed model evaluation guidelines, in which NSE values below 0.50 are generally regarded as unsatisfactory (Moriassi et al., 2007), ensuring that workflows with executable outputs but clearly unsatisfactory hydrological performance were not treated as successful.

Runs that failed to satisfy any of the above conditions were not counted as complete successes. Specifically, runs with unresolved syntax errors, runtime errors, missing evaluation metrics, infinite logical loops, or timeout exceedance were classified as failed runs. Runs that completed the full workflow but yielded $NSE < 0.50$ were also excluded from the complete success count and annotated as weak-performance cases requiring human expert review. Therefore, the reported complete success rate reflects both autonomous workflow completion and a minimum hydrological validity requirement, while detailed NSE values are separately reported for the hydrological performance analysis.

3.2. Phase 2: Hydrologic performance evaluation

Given the high training time cost of the 531-basin experiments, GPT-5.5 was used as the fixed LLM backbone owing to its stable execution and relatively low tool-call overhead during cross-LLM evaluation.

3.2.1. CAMELS dataset

As presented in Table 2, we employ the CAMELS dataset to evaluate the hydrologic performance (Addor et al., 2017; Newman et al., 2015). We selected a subset of 531 catchments across the contiguous United States (US), covering a wide spectrum of climatic zones and hydrologic regimes, as shown in Fig. 7.

The data inputs for the models comprised dynamic meteorological forcings and static catchment attributes. For dynamic forcings, the data preprocessing agent used daily meteorological time-series data from the extended Maurer forcing dataset, including precipitation, maximum and minimum air temperatures, and other meteorological variables required for rainfall-runoff simulation (Kratzert, 2019). The target variable for all CAMELS modelling tasks was daily streamflow, expressed as specific discharge (mm/day), the standard unit commonly used in comparative large-sample hydrology and CAMELS benchmark studies. The CAMELS experiments were formulated as rainfall-runoff simulation tasks, in which meteorological forcings were used to simulate streamflow without relying on observed discharge autoregression. To enable spatial generalisation, we incorporated 27 static catchment attributes derived from the CAMELS dataset (Kratzert et al., 2019). These attributes encompassed five physical categories: topographic characteristics, climatic indices, soil properties, vegetation cover, and geological features. These static descriptors allow the model to differentiate rainfall-runoff behaviours based on catchment heterogeneity rather than through spatial coordinates alone. For data partitioning, we followed the standard CAMELS benchmark-compatible protocol: the period from 1 October 1999 to 30 September 2008 was used as the calibration period, and the period from 1 October 1989 to 30 September 1999 was used as the benchmark evaluation period. For neural-network training, the calibration period was further divided into an internal training period from 1 October 1999 to 30 September 2006 and an internal validation period from 1 October 2006 to 30 September 2008, while the benchmark evaluation period was kept strictly independent and reserved exclusively for final testing.

3.2.2. Benchmark models

To evaluate the performance of HydroAIM, we employed three

Table 4
Configuration of ablation studies.

Experimental Group	Multi-Agent Architecture	SAW library	Closed-loop Feedback	Complete Workflow
HydroAIM	✓	✓	✓	✓
w/o Multi-agent	×	✓	✓	✓
w/o SAW library	✓	×	✓	✓
w/o Feedback	✓	✓	×	✓
w/o Workflow	✓	✓	✓	×

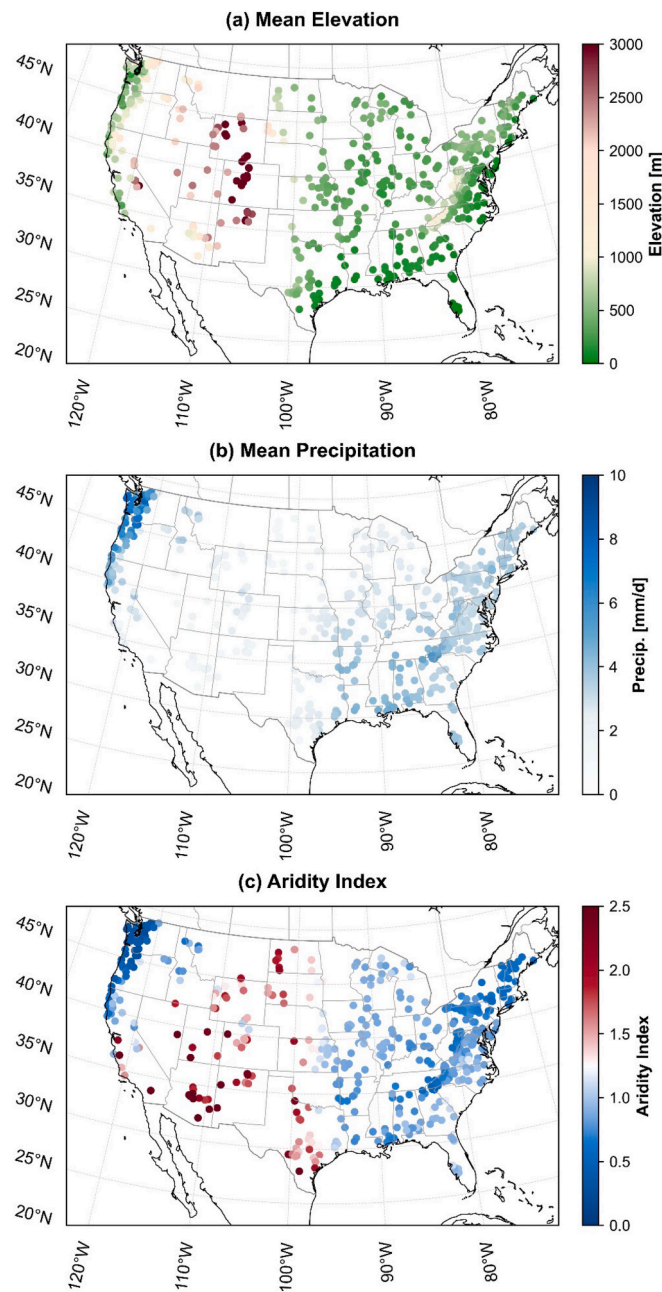


Fig. 7. Spatial distribution of the 531 study catchments across the contiguous United States. The maps display the hydro-climatic and topographic characteristics of the Catchment Attributes and Meteorology for Large-sample Studies (CAMELS) dataset: (a) mean catchment elevation (m), showing the topographic gradient from the mountainous west to the lower east; (b) mean daily precipitation (mm/day), with dark blue indicating wetter regions; and (c) aridity index (ratio of mean potential evapotranspiration to mean precipitation), wherein red and blue tones represent water- (dry; AI > 1) and energy-limited (wet; AI < 1) catchments, respectively. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

distinct benchmark categories: an established physical benchmark, a published deep learning benchmark, and a manually implemented LSTM baseline.

Established physical benchmark. We selected the Sacramento Soil Moisture Accounting model coupled with the Snow-17 snow routine (SAC-SMA/Snow-17) as the representative process-based benchmark. Instead of recalibrating SAC-SMA in this study, we used the published

CAMELS benchmark model outputs available from the HydroShare CAMELS benchmark archive. This archive contains hydrological model outputs calibrated with the same Maurer forcing data over the same calibration period, from 1 October 1999 to 30 September 2008, and provides model simulations for the validation period from 1 October 1989 to 30 September 1999. These basin-wise calibrated SAC-SMA outputs were generated using the standard CAMELS benchmark protocol (Newman et al., 2017). Therefore, the comparison with HydroAIM was conducted over the same benchmark evaluation period, using basin-level NSE values to assess the performance of the agent-generated, data-driven models relative to this expert-calibrated, process-based benchmark.

Published deep-learning benchmark. As a widely cited reference for data-driven CAMELS rainfall-runoff modelling, the published LSTM benchmark with static catchment attributes reports a median NSE of approximately 0.72 (Kratzert et al., 2019). This benchmark represents a regional/global LSTM trained across multiple basins, rather than independently trained single-basin LSTM models. Therefore, the published LSTM benchmark was used only as a reference for evaluating the global modelling results of HydroAIM.

Human-expert baseline (Human-LSTM). To rigorously validate the code reliability of the agent, we constructed a control group by manually implementing a standard LSTM model. Notably, to ensure a fair comparison of ‘coding capability’, this manual baseline utilises the same as suggested by the agent’s initial task analysis. By explicitly directing the agent to adopt the LSTM architecture for this specific validation task, we isolated the effect of code implementation by using the same preprocessing, temporal split, input-output definition, sequence length, target variable, and hyperparameters, enabling verification that the agent-generated code was functionally equivalent to the human-expert code.

3.2.3. Experimental design

The experimental evaluation is divided into two progressive phases: performance and capability expansion. These phases were designed to sequentially validate the fundamental reliability of HydroAIM and explore its boundaries when guided by advanced prompting strategies.

Performance evaluation. This initial phase was strictly confined to local modelling scenarios to ascertain the reliability of the code, hydrologic consistency, and comparative performance of the proposed approach against traditional baselines. The experiments were formulated as rainfall-runoff simulation tasks, using meteorological forcings to simulate streamflow without observed discharge autoregression. In experiment I (verification of code reliability), we conducted a strict control comparison between the HydroAIM-generated code and manual-LSTM baseline. By constraining both implementations to the same preprocessing procedure, data split, input-output definition, sequence length, and hyperparameters, the parity in the metrics confirms the agent’s ability to generate bug-free, expert-level code. Building on this, in experiment II (sensitivity to input sequence length), we evaluated how the temporal receptive field affects local modelling performance by instructing the agent to construct models with varying sequence lengths (30, 90, 180, 270, and 365 days). This experiment was designed to assess the sensitivity of HydroAIM-generated workflows to temporal configuration. Finally, in experiment III (benchmarking against physical models), we compared the HydroAIM-generated local modelling workflows with the established SAC-SMA/Snow-17 benchmark over the same CAMELS evaluation period. To assess robustness, HydroAIM was allowed to autonomously select the model architecture and was executed across five independent runs, with the resulting basin-level NSE distributions reported for comparison.

Capability expansion. Although the preceding evaluations focused on the approach’s primary design for local modelling, in this phase, we tested its extensibility to complex scenarios through predefined expert guidance. In experiment IV (prompt-guided global modelling), we introduced a global modelling workflow to test zero-shot task

adaptation. Drawing upon the open “Agent Skills” standard (Anthropic, 2025), a suite of modular reference scripts, spanning from data merging and multi-scale normalisation to distributed evaluation, was integrated into the SAW library as an on-demand skill set. This experiment verifies whether the agent can strictly follow expert prompts to autonomously retrieve and execute these specific skills, thereby orchestrating the 531-basin spatial generalisation task.

3.2.4. Evaluation metric

To quantitatively assess the predictive performance of HydroAIM and the benchmark models, we adopted the Nash-Sutcliffe efficiency (NSE) coefficient as the primary evaluation metric. As a standard in hydrologic modelling, the NSE determines the relative magnitude of the residual variance compared with the measured data variance. This is mathematically defined as follows:

$$NSE = 1 - \frac{\sum_{t=1}^T (Q_{obs}^t - Q_{sim}^t)^2}{\sum_{t=1}^T (Q_{obs}^t - \bar{Q}_{obs})^2} \quad (8)$$

where Q_{obs}^t and Q_{sim}^t represent the observed and simulated discharges at time step t , respectively, and $\bar{Q}_{obs}$ is the mean observed discharge. The NSE value ranges from $-\infty$ to 1, where an NSE of 1 indicates a perfect match between the modelled and observed data, and an NSE of 0 indicates that the model predictions are only as accurate as the mean of the observed data. Notably, compared with standard regression metrics, such as the mean squared error (MSE), the NSE is dimensionless, facilitating performance comparisons across basins with vastly different flow magnitudes.

4. Results

4.1. Results of the capability of the approach

4.1.1. Adaptability across LLM backbones

As outlined above, to verify the approach’s capabilities and explicitly

address the first core scientific question, we deployed the approach on representative LLMs: GPT-5.5, Claude-4.5-sonnet, Gemini-2.5-pro, Deepseek-v3.2, and Qwen-plus. The experimental results demonstrated that the approach exhibited high adaptability across the selected backbones. As shown in Fig. 8, despite the differences in their underlying training data and alignment strategies, all five models successfully comprehended the expert task prompts and executed the complete modelling pipeline with high consistency. These consistent performance metrics across diverse backbones confirm the generalisability of the proposed automated workflow, indicating that its effectiveness is independent of any specific proprietary model architecture.

4.1.2. Ablation study on modules

To validate the scientific necessity of each component within HydroAIM and explicitly address the second core scientific question, a systematic ablation study was conducted using DeepSeek-v3.2 as the fixed reasoning kernel. We evaluated the task success rates of the four architectural variants across five distinct task categories, and the comparative performance is illustrated in Fig. 9. The results provided critical insights into the dependency between agent autonomy and engineering constraints.

The most significant effect was observed in the workflow ablation (w/o workflow) group, where the removal of expert-task workflow constraints resulted in complete failure, with the success rate dropping to 0% across all tasks (Fig. 9d). This outcome highlights that the primary bottleneck for LLMs in complex engineering applications is not a deficiency in domain knowledge but a lack of autonomous workflow orchestration. Without the constraints of a structured workflow, even with extensive knowledge, agents cannot maintain logical consistency across long-term sequences, rendering autonomous modelling operationally infeasible. Qualitative inspection of the failed runs revealed several representative patterns of workflow collapse. First, some runs terminated prematurely after the data-preprocessing or model-construction stage, treating the completion of a single-agent procedure as the completion of the entire workflow. Second, some runs repeatedly invoked similar tools, such as file reading or code checking, without

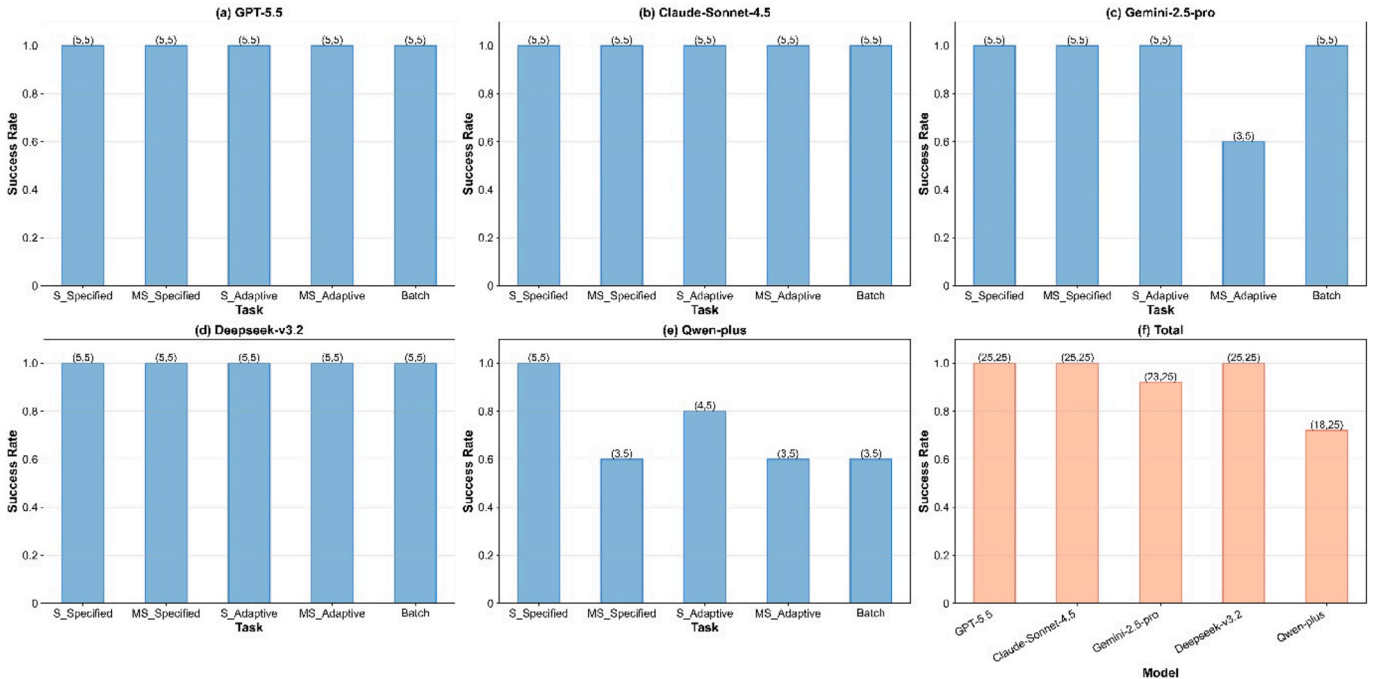


Fig. 8. Evaluation of the generality of HydroAIM across representative large language model (LLM) backbones. Panels (a–e) present task-specific success rates for GPT-5.5, Claude-Sonnet-4.5, Gemini-2.5-Pro, DeepSeek-v3.2, and Qwen-Plus, respectively. Panel (f) summarises the total number of successful runs for each LLM across all task groups. The results show that HydroAIM maintains stable execution performance across different LLM backbones, indicating that the expert task workflow is not tied to a specific proprietary model.

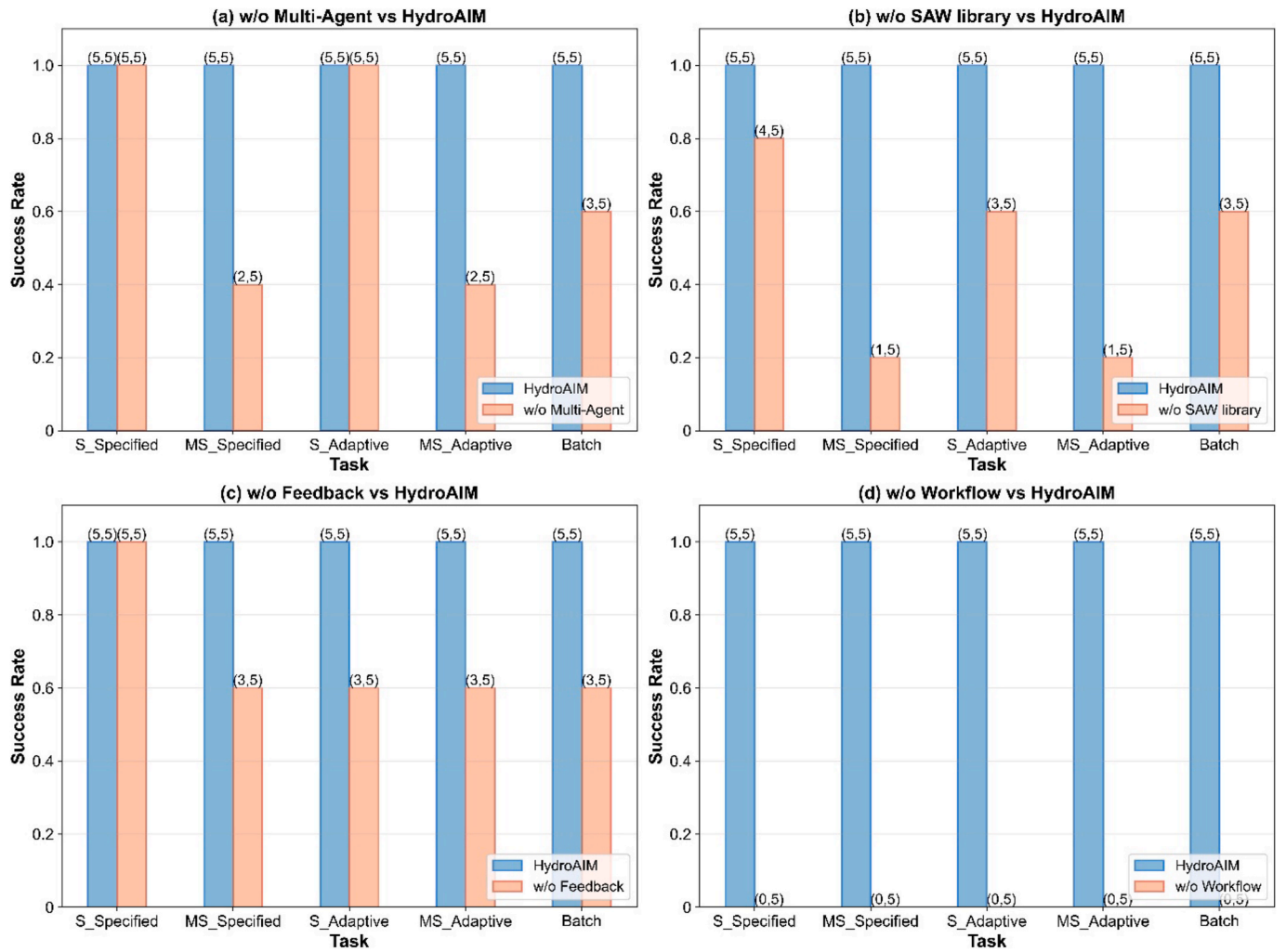


Fig. 9. Comparisons of task success rates between the complete HydroAIM approach and four ablation variants. The blue bars represent the complete HydroAIM, whereas the orange bars represent the specific ablation variant. The evaluation encompasses five task categories: univariate/specified (S-Specified), multivariate/specified (MS-Specified), univariate/adaptive (S-Adaptive), multivariate/adaptive (MS-Adaptive), and batch modelling. The subplots correspond to: (a) architecture ablation (w/o multi-agent), (b) constraint ablation (w/o SAW library), (c) mechanism ablation (w/o feedback), and (d) workflow ablation (w/o workflow). (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

producing a new workflow state, eventually leading to timeouts or tool-call limits. Third, after context pruning or message summarisation, key task states were sometimes lost, making it difficult for subsequent agents to recover the modelling objective, data paths, or configuration parameters. Fourth, incomplete handoff files, such as missing JSON summaries, file paths, or parameter records, prevented downstream agents from continuing the pipeline. These failure patterns suggest that the expert-task workflow acts as a global process constraint by defining stage boundaries, stopping criteria, mandatory outputs, and inter-agent handoff requirements, thereby preventing agents from mistaking local subtask completion for end-to-end modelling success.

The architecture ablation (w/o multi-agent) group demonstrates the critical necessity of role decoupling (Fig. 9a). Although the single-agent architecture achieved a 100% success rate in simple tasks, its performance declined significantly to 40% in complex multivariate scenarios. This performance gap indicates that a single agent encounters cognitive limitations while simultaneously managing high-dimensional data processing and complex model construction. By decoupling different functions, the multi-agent architecture effectively reduces context interference, ensuring that the stability of the approach is maintained with increasing task complexity.

The integration of the algorithm constraints and feedback mechanisms proved essential for ensuring both architectural and operational

robustness. The constraint ablation (w/o SAW library) approach exhibited an obvious performance dichotomy (Fig. 9b); although the agent maintained a success rate of 80% on standard LSTMs, the performance declined to 20% for advanced models (e.g. Crossformer). This indicates that without external structural references, LLMs struggle to implement complex deep-learning architectures from scratch.

The mechanism ablation (w/o feedback) group showed uniform performance degradation across almost all tasks (Fig. 9c), suggesting that the absence of a self-correction loop makes the approach vulnerable to stochastic errors, such as syntax bugs or application programming interface (API) timeouts, which is otherwise corrected through iterative debugging. In summary, the robustness of HydroAIM is an emergent property that results from the synergistic integration of structured workflows, collaborative agents, algorithm constraints, and self-correction mechanisms.

4.2. Results of performance evaluation

In this section, HydroAIM is evaluated from three perspectives: the functional consistency of the generated code relative to a Human-LSTM baseline (experiment I), the sensitivity of local modelling performance to input sequence length (experiment II), and the comparative performance of autonomously generated local modelling workflows against

established physical benchmarks (experiment III).

4.2.1. Verification of code reliability

To validate the functional consistency of the code generated by HydroAIM, we employed an LSTM model that utilises dynamic time-series variables to conduct batch streamflow modelling across all 531 basins and a manual LSTM for comparison. As detailed in Section 3.2.3, both workflows were constrained to the same preprocessing, temporal split, input–output definition, model configuration, hyperparameters, and evaluation script, thereby focusing the comparison on whether the agent-generated code could reproduce the manually implemented workflow.

The results of the comparison shown in Fig. 10 demonstrate a high degree of consistency between the autonomous work of the agent and manual implementation. The scatter plot reveals a tight linear relationship along the 1:1 line with a coefficient of determination (R^2) of 0.88. This confirms that HydroAIM correctly assembled the deep learning process, implementing critical components such as tensor construction, backpropagation, and loss calculation without logic errors. Minor basin-level deviations persisted, reflecting the sensitivity of independently trained local LSTMs to limited basin-specific training data and to stochastic optimisation rather than to systematic implementation errors.

4.2.2. Sensitivity to input sequence length

The input sequence length defines the temporal receptive field of the deep-learning model and determines the extent of antecedent meteorological forcings available for rainfall-runoff simulation. To evaluate the sensitivity of HydroAIM-generated local workflows to temporal configuration, we instructed the agent to construct LSTM-based models with sequence lengths of 30, 90, 180, 270, and 365 days.

As shown in Fig. 11, HydroAIM maintained generally stable

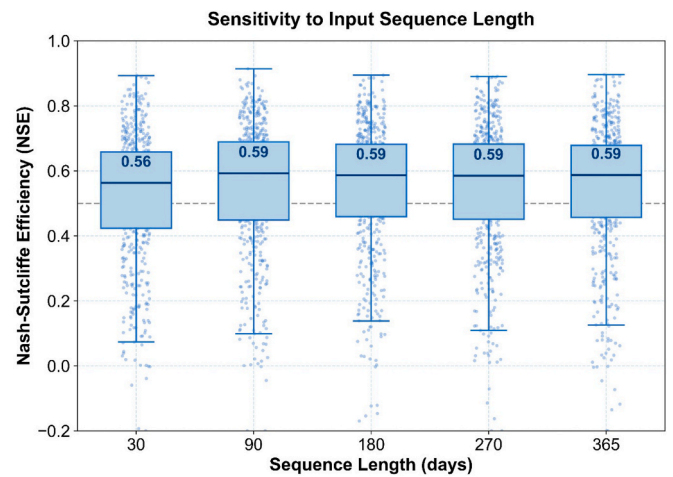


Fig. 11. Sensitivity analysis of HydroAIM-generated local LSTM workflows to input sequence length. Boxplots and jittered points show basin-level NSE distributions across the 531 CAMELS benchmark basins for sequence lengths of 30, 90, 180, 270, and 365 days. The dashed horizontal line indicates NSE = 0.50. Extreme negative NSE values were clipped at -0.2 for visual clarity. The comparable distributions indicate that the local workflow was relatively stable across temporal-window configurations.

performance across different input windows. Although the median NSE varied slightly among the tested configurations, the overall distributions were comparable once the sequence length exceeded 90 days. This suggests that the generated workflow was not highly sensitive to a single temporal window selected under the benchmark-compatible local CAMELS setting.

The 365-day window was retained in subsequent local modelling experiments as a literature-supported full-annual configuration. This setting provides the model with access to a complete seasonal hydrological cycle and is consistent with common LSTM rainfall-runoff modelling practices. Therefore, this experiment was primarily used to verify the robustness of HydroAIM-generated workflows to temporal configuration, rather than to identify a universally optimal input sequence length.

4.2.3. Benchmarking against physical models

To provide a benchmark-compatible comparison with an established physical model, we used the published SAC-SMA/Snow-17 outputs from the CAMELS benchmark archive. Both SAC-SMA/Snow-17 and HydroAIM were evaluated over the same CAMELS benchmark evaluation period from 1 October 1989 to 30 September 1999. HydroAIM was allowed to autonomously select the model architecture and executed five times to assess the stability of the generated local modelling workflows.

As shown in Fig. 12, all five HydroAIM runs produced similar basin-level NSE distributions. In all runs, the agent selected an LSTM architecture for local modelling. The HydroAIM local models achieved median NSE values of approximately 0.58–0.59, which were close to but slightly lower than the SAC-SMA/Snow-17 median NSE of 0.61. This result indicates that HydroAIM can autonomously construct stable local rainfall-runoff modelling workflows, although it does not imply that independently trained local LSTMs universally outperform basin-wise calibrated physical models.

The slightly lower local LSTM performance should be interpreted in relation to the benchmark-compatible experimental design. To ensure direct comparability with SAC-SMA/Snow-17 and the subsequent global modelling experiment, the local models were trained only during the standard CAMELS calibration period, while the independent evaluation period was kept unchanged. This choice deliberately prioritised protocol consistency over maximising local LSTM accuracy. Previous CAMELS

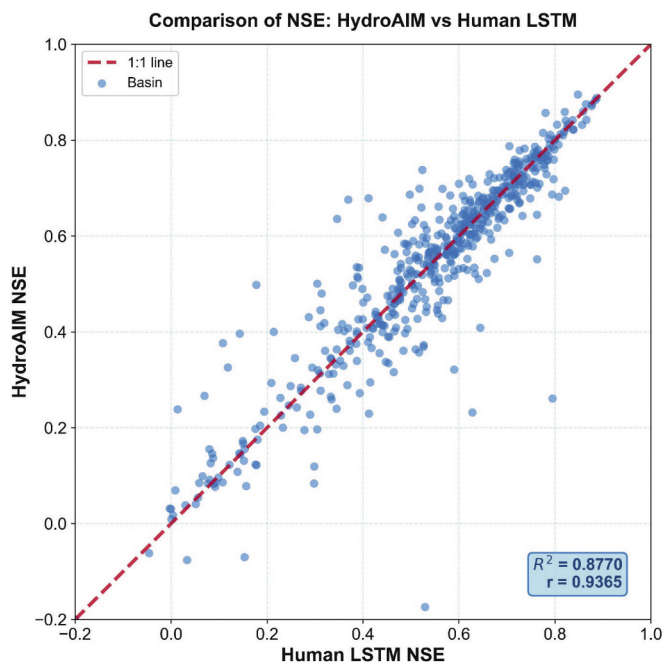


Fig. 10. Comparison of the performance between HydroAIM and manual long short-term memory (LSTM). Each point represents the Nash-Sutcliffe efficiency (NSE) score of a single basin. The red dashed line denotes the 1:1 identity line. The high coefficient of determination ($R^2 = 0.88$) confirms the functional correctness of the agent-generated code, whereas the slight distribution of points above the diagonal reflects the enhanced robustness of the agent's standardised data preprocessing pipeline. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

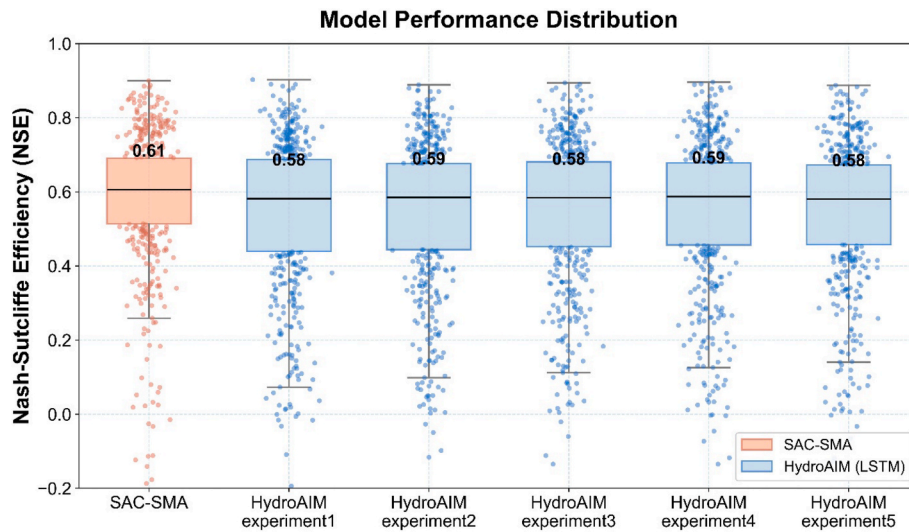


Fig. 12. Boxplot comparison of basin-level Nash-Sutcliffe efficiency (NSE) distributions between the Sacramento Soil Moisture Accounting model coupled with the Snow-17 snow routine (SAC-SMA/Snow-17) benchmark and five independent HydroAIM local modelling runs. HydroAIM was allowed to autonomously select the model architecture in each run and selected LSTM in all five runs. Boxplots and jittered points show the NSE distributions across the 531 CAMELS benchmark basins.

LSTM studies used longer basin-specific calibration records for local LSTM training and noted that reducing the calibration period can limit the information available for learning rainfall-runoff dynamics (Kratzert et al., 2018). Therefore, the local comparison should be viewed as a conservative assessment of automated workflow construction under a strict benchmark protocol.

These results further motivate the global modelling experiment presented in Section 4.3. Unlike independently trained local models, global LSTM modelling can exploit cross-basin information and static catchment attributes, a key advantage of large-sample LSTM rainfall-runoff modelling.

4.3. Results of capability expansion

This section evaluates the extensibility of HydroAIM to more complex scenarios, focusing on prompt-guided adaptation from local modelling to large-sample global rainfall-runoff simulation (Experiment

IV).

4.3.1. Prompt-guided global rainfall-runoff simulation

To evaluate zero-shot task adaptation, in this experiment, we analysed the capability of the approach to orchestrate a large-scale spatial generalisation task across 531 basins. In contrast to local rainfall-runoff simulation, this task necessitates the autonomous execution of sophisticated engineering pipelines. This requires the agent to seamlessly integrate static attributes and handle complex workflows, encompassing massive data merging, chronological time-series splitting, differentiated normalisation strategies, and distributed evaluation using the standard CAMELS benchmark periods. The execution results empirically validate the efficacy of the prompt-guided paradigm. Rather than relying on static hard coding, HydroAIM successfully parses CoT prompts to autonomously retrieve and execute the required modular scripts from the SAW library as on-demand skills. Driven by the task requirements, the agent selects the LSTM architecture and instantiates the complete

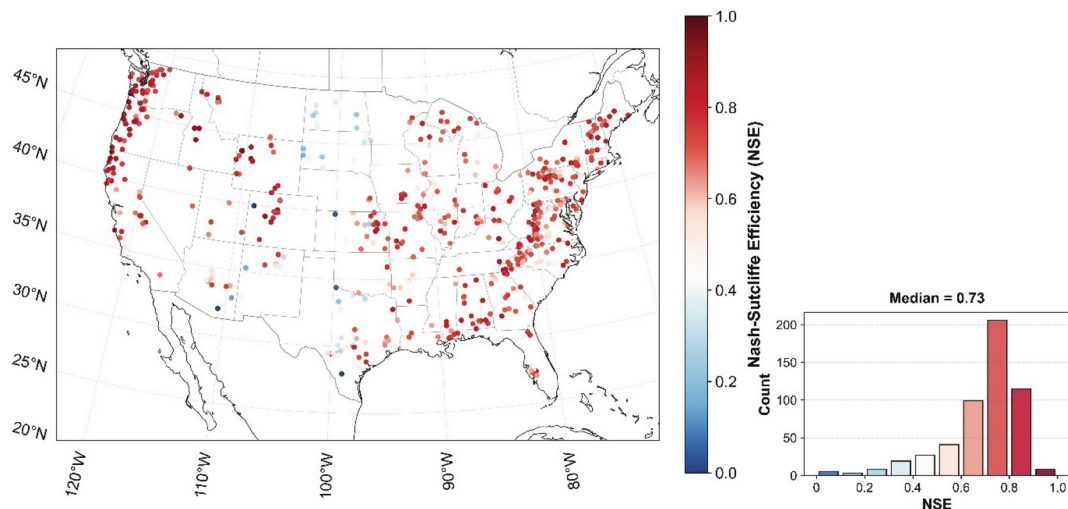


Fig. 13. Spatial distribution and frequency distribution of hydrological performance for the HydroAIM-generated global rainfall-runoff simulation model. The global LSTM model was constructed under the benchmark-compatible CAMELS setting using meteorological forcings and static catchment attributes. The map shows the basin-level Nash-Sutcliffe efficiency (NSE) across the 531 CAMELS benchmark basins, and the histogram summarises the corresponding NSE distribution. The global model achieved a median NSE of 0.73, indicating benchmark-level performance for large-sample rainfall-runoff simulation.

workflow. As illustrated in Fig. 13, this approach produced valid hydrological simulations, with a median NSE of 0.73. This success, without manual modification of intermediate outputs, demonstrates that the agent can precisely interpret complex human directives, effectively bridging the gap between modular skill libraries and dynamic large-scale deployment requirements, while achieving performance comparable to that of widely cited CAMELS LSTM benchmarks.

In addition to comparing HydroAIM against the published LSTM benchmark value, Fig. 14 compares the spatial distribution of HydroAIM and SAC-SMA. The spatial difference map further indicates that HydroAIM achieved broadly competitive performance across diverse hydroclimatic regions, with positive NSE differences observed in many catchments. This comparison confirms that promptly guided skill execution can autonomously produce simulation-grade global models that reach benchmark-level performance.

5. Discussions

The results should be interpreted as evidence that LLM-based agents can support the automated construction and execution of hydrological modelling workflows when constrained by expert workflows, standardised algorithmic references, structured tool schemas, and executable feedback. HydroAIM does not imply that the LLM directly performs hydrological modelling. Instead, hydrological forecasting and simulations are generated by deep learning models and data-processing pipelines, while the LLM provides workflow reasoning, tool orchestration, and iterative correction.

The autonomous execution setting used in this study operates without human intervention unless an uncorrectable error occurs, thereby minimising the need for human hydrological expertise. Full automation improves scalability, but expert judgement remains necessary for task formulation, data preprocessing, model selection, diagnosis of weak performance, and assessment of physically plausible hydrographs. For example, in one LSTM-based global modelling run, weak performance was traced to a data-loading configuration that did not provide target-day meteorological forcings, representing a task-formulation issue that required expert review.

This need for expert oversight was also evident in prompt-reinforcement trials. When prompts explicitly emphasised data volume and architectural capacity, the agents sometimes shifted from LSTM to DLinear in local tasks or to Crossformer in global tasks. However, under

the CAMELS rainfall-runoff simulation setting, these prompt-induced choices did not necessarily improve performance and were often inferior to the LSTM-based workflow. This suggests that prompt reinforcement can effectively redirect the agent's attention, but it does not guarantee scientifically appropriate modelling decisions if the prompt encodes unsuitable assumptions. Therefore, prompts should be treated as modelling constraints that require expert design and review rather than as neutral instructions.

Reproducibility is also more complex for LLM-based modelling than for fixed-code pipelines. Although HydroAIM reduces hallucination through the SAW library, strict tool schemas, and closed-loop execution feedback, the outputs may still depend on model versions, decoding settings, prompt wording, tool schema design, and API updates. Model-version changes were already encountered in the supplementary CAMELS experiments, where newer Qwen and DeepSeek versions were used than those used in the initial experiments. Therefore, reproducing HydroAIM requires the recording of model identifiers, prompt templates, tool schemas, execution logs, generated artefacts, datasets, and evaluation scripts.

Practical cost is another limitation. HydroAIM introduces additional LLM calls for task analysis, code generation, tool invocation, debugging, and result interpretation, increasing token usage and wall-clock time. Representative runs were therefore logged for token usage, execution time, and estimated API cost across LLM backbones; details are provided in Appendix A. These estimates are intended as practical reference values rather than formal cost benchmarks.

Finally, the current implementation focuses on validating agentic workflow automation rather than building a physically complete hydrological modelling system. Explicit conservation constraints, physics-guided learning, and basin-specific fine-tuning have not yet been incorporated (Chadalawada et al., 2020; Kratzert et al., 2019; Read et al., 2019). Future work should extend the SAW library with physics-guided and differentiable hydrological modules, improve memory management for long workflows, and integrate expert review more tightly into autonomous modelling pipelines (Shen et al., 2023a).

6. Conclusion

In this study, we grounded LLMs in hydrological modelling by employing HydroAIM to fill the gaps between knowledge interactions and quantitative modelling in LLM applications. The results verify the

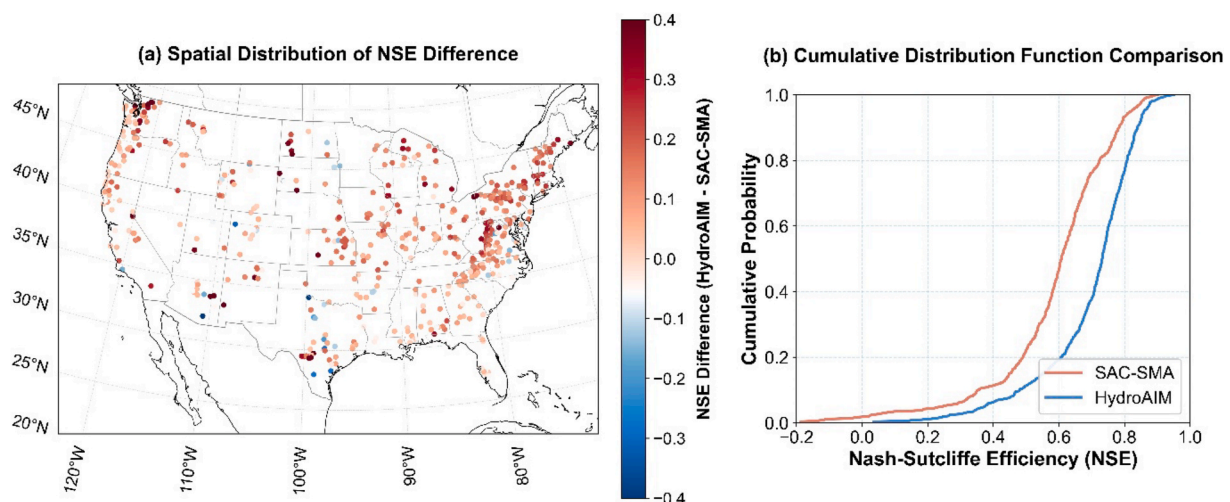


Fig. 14. Performance comparison between the HydroAIM-generated global model and the Sacramento Soil Moisture Accounting (SAC-SMA) benchmark. (a) Spatial distribution of basin-level NSE differences, calculated as NSE of HydroAIM minus NSE of SAC-SMA. Red regions indicate catchments where the HydroAIM-generated model achieved higher NSE values than SAC-SMA, whereas blue regions indicate lower NSE values. (b) Cumulative distribution function (CDF) curves of basin-level NSE values for HydroAIM and SAC-SMA, comparing a single global data-driven model with the basin-wise calibrated physical benchmark. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

feasibility and effectiveness of the “expert experience + LLM-based agent” collaboration paradigm in hydrologic modelling tasks. Using HydroAIM, we confirm that, under the constraints of mechanistic workflows, general LLMs can achieve highly reliable autonomous execution. Across 125 diverse modelling tasks utilising five different LLMs, the approach achieved an impressive execution success rate of 92.8%, demonstrating the adaptability and robustness of HydroAIM in handling complex professional modelling processes. Furthermore, ablation experiments indicated that long-horizon task planning is the core bottleneck restricting LLMs from executing quantitative tasks. Removing the expert agentic workflow led to a complete system failure, with the success rate decreasing precipitously to 0%. The ablation of other architectural components results in significant performance degradation. This comprehensive evaluation indicated that the synergistic integration of the MCP-based multi-agent architecture, algorithmic and expert-experience constraints, and a closed-loop execution environment is indispensable for successful task execution. Finally, we showed that HydroAIM can effectively mitigate the “weak transferability” issue common in traditional manual deep-learning modelling. Deployed across 531 catchments in the CAMELS dataset without human intervention, the models, autonomously orchestrated by the agents, achieved median NSE values of 0.58 and 0.73 in local and global modelling, respectively. This demonstrates that HydroAIM can achieve usable modelling performance.

Despite these promising results, several limitations remain; HydroAIM provides a foundational, scalable prototype for automating hydrological deep learning. It not only significantly lowers the technical barrier to LLM applications in hydrology but also establishes a flexible technical pathway: future process-based and differentiable models can

be seamlessly integrated as workflows and tools through structural decomposition, thereby providing a novel methodological reference for more efficient research.

CRediT authorship contribution statement

Yingjia Li: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Shiruo Hu:** Writing – review & editing, Validation, Methodology, Investigation. **Feng Zhang:** Writing – review & editing, Validation, Software, Investigation. **Xinpeng Yu:** Writing – review & editing, Software, Data curation. **Wei Luo:** Writing – review & editing, Methodology, Formal analysis. **Dingxiao Liu:** Visualization, Investigation. **Bin Xu:** Writing – review & editing, Resources. **Jianshi Zhao:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the National Natural Science Foundation of China (No. 52539001) and Shenzhen Science and Technology Program (No. AI20260020).

Appendix 1

Appendix A. . Representative successful runs used for cost and efficiency estimation

The records were randomly sampled from successful runs for each LLM-task combination. Runs that succeeded after error correction are retained, so retry overhead is reflected in the API calls, tool calls, token usage, and execution time.

Table A1
Representative successful runs used for cost and efficiency estimation.

LLM backbone	Task group	LLM API calls	MCP tool calls	Wall-clock time	Tokens (input)	Tokens (output)	Total tokens	cost (USD)
Qwen-plus	MS_Specified	52	56	41m38s	1,440,954	25,962	1,466,916	0.085
Qwen-plus	S_Specified	52	68	10m54s	1,365,413	23,318	1,388,731	0.081
Qwen-plus	MS_Adaptive	54	58	50m38s	1,564,096	31,996	1,596,092	0.093
Qwen-plus	S_Adaptive	30	35	7m50s	760,810	17,312	778,122	0.046
Qwen-plus	Batch	53	70	14m40s	1,500,268	32,709	1,532,977	0.090
Deepseek-v3.2	MS_Specified	59	60	39m13s	1,588,255	28,059	1,616,314	0.099
Deepseek-v3.2	S_Specified	33	31	5m30s	1,073,370	16,099	1,089,469	0.066
Deepseek-v3.2	MS_Adaptive	66	60	1h48m	1,331,629	43,209	1,374,838	0.088
Deepseek-v3.2	S_Adaptive	53	56	8m35s	1,027,761	25,555	1,053,316	0.066
Deepseek-v3.2	Batch	71	69	7m28s	2,245,806	23,718	2,269,524	0.135
Claude-Sonnet-4.5	MS_Specified	31	38	42m46s	541,292	13,125	554,417	1.821
Claude-Sonnet-4.5	S_Specified	44	57	27m5s	847,073	19,481	866,554	2.833
Claude-Sonnet-4.5	MS_Adaptive	27	38	53m6s	568,210	9,144	577,354	1.842
Claude-Sonnet-4.5	S_Adaptive	39	53	17m0s	671,191	17,049	688,240	2.269
Claude-Sonnet-4.5	Batch	39	71	26m40s	772,042	15,613	787,655	2.550
Gemini-2.5-pro	MS_Specified	50	45	57m17s	1,309,260	35,578	1,344,838	3.807
Gemini-2.5-pro	S_Specified	56	64	22m1s	1,380,034	31,734	1,411,768	3.926
Gemini-2.5-pro	MS_Adaptive	63	59	58m54s	1,436,952	33,120	1,470,072	4.089
Gemini-2.5-pro	S_Adaptive	36	44	10m14s	739,454	20,094	759,548	2.150
Gemini-2.5-pro	Batch	55	71	19m23s	1,536,871	33,010	1,569,881	4.337
GPT-5.5	MS_Specified	25	30	57m27s	577,827	10,128	587,955	3.193
GPT-5.5	S_Specified	28	35	14m48s	565,716	7,516	573,232	3.054
GPT-5.5	MS_Adaptive	72	81	46m21s	1,538,507	14,581	1,553,088	8.130
GPT-5.5	S_Adaptive	26	33	11m29s	480,869	5,797	486,666	2.578
GPT-5.5	Batch	48	62	26m19s	1,023,233	16,266	1,039,499	5.604

Appendix B. . Representative CAMELS replication experiment

To improve reproducibility, a representative capability replication experiment was conducted on the public CAMELS dataset. Five representative task groups were tested, corresponding to multivariate/specified simulation, univariate/specified forecasting, multivariate/adaptive simulation, univariate/adaptive forecasting, and batch simulation. Five LLM backbones were evaluated: Qwen3.7-Plus, DeepSeek-v4-Pro, Claude-Sonnet-4.5, Gemini-2.5-Pro, and GPT-5.5. Qwen3.7-Plus and DeepSeek-v4-Pro were used because the earlier Qwen-Plus and DeepSeek-v3.2 versions were no longer available after model-version updates. All LLM backbones completed the representative CAMELS workflows. Because not all CAMELS basins are equally suitable for single-basin modelling, runs with NSE slightly below 0.50 were manually checked and annotated as weak-performance but executable workflows.

Table B1

Representative CAMELS replication task configurations and completion results.

Task group	Task description	Model setting	Result
MS_Specified_Simulation	Single-basin multivariate rainfall-runoff simulation	Specified LSTM	Completed by all five LLMs
S_Specified_Forecasting	Single-basin univariate forecasting	Specified LSTM	Completed by all five LLMs
MS_Adaptive_Simulation	Single-basin multivariate rainfall-runoff simulation	Adaptive model selection	Completed by all five LLMs
S_Adaptive_Forecasting	Single-basin univariate forecasting	Adaptive model selection	Completed by all five LLMs
Batch_Simulation	Multi-basin rainfall-runoff simulation	Specified LSTM	Completed by all five LLMs

In addition to the five representative task groups, an extra S_Specified_Simulation stress test was conducted. In this task, the agents were explicitly instructed to perform rainfall-runoff simulation using only a single forcing variable. Although all LLMs completed the workflow, this setting is hydrologically inappropriate and produced poor performance. Therefore, this task was not treated as a valid hydrological modelling configuration, but was used as a prompt-sensitivity case showing that agents may faithfully execute scientifically unsuitable user instructions.

Appendix C. Supplementary data

Supplementary data to this article can be found online at <https://doi.org/10.1016/j.jhydrol.2026.136358>.

Data availability

Phase 2 CAMELS data is publicly available. Phase 1 Dadu River operational data is restricted due to infrastructure security and NDAs.

References

- Achiam, J. et al., 2023. Gpt-4 technical report. arXiv preprint arXiv:2303.08774.
- Addor, N., Newman, A.J., Mizukami, N., Clark, M.P., 2017. The CAMELS data set: catchment attributes and meteorology for large-sample studies. *Hydrol. Earth Syst. Sci.* 21 (10), 5293–5313. <https://doi.org/10.5194/hess-21-5293-2017>.
- Alansari, A., Luqman, H., 2025. Large Language Models Hallucination: a Comprehensive Survey. *ArXiv*, abs/2510.06265.
- Anthropic, 2024a. The Claude 3 Model Family: Opus, Sonnet, Haiku, Anthropic Technical Report.
- Anthropic, 2024b. Model Context Protocol.
- Anthropic, 2025. Agent Skills: The open standard for AI capabilities.
- Boiko, D.A., MacKnight, R., Gomes, G., 2023. Emergent Autonomous Scientific Research Capabilities of Large Language Models. *ArXiv*, abs/2304.05332.
- Chadalawada, J., Herath, H.M.V.V., Babovic, V., 2020. Hydrologically informed machine learning for rainfall-runoff modeling: a genetic programming-based toolkit for automatic model induction. *Water Resour. Res.* 56 (4).
- Chan, J.S. et al., 2024. Mle-bench: Evaluating machine learning agents on machine learning engineering. arXiv preprint arXiv:2410.07095.
- Dong, Y., Jiang, X., Jin, Z., Li, G., 2024. Self-collaboration code generation via chatgpt. *ACM Trans. Softw. Eng. Methodol.* 33 (7), 1–38.
- Eythorsson, D., Clark, M., 2025. Toward automated scientific discovery in hydrology: the opportunities and dangers of AI augmented research frameworks. *Hydrol. Process.* 39 (1). <https://doi.org/10.1002/hyp.70065>.
- Fang, K., Kifer, D., Lawson, K., Feng, D., Shen, C., 2021. The data synergy effects of time-series deep learning models in hydrology.
- Fatouros, G. et al., 2025. Towards Conversational AI for Human-Machine Collaborative MLOps.
- Feng, J. et al., 2025. M³Builder: A Multi-Agent System for Automated Machine Learning in Medical Imaging. arXiv preprint arXiv:2502.20301.
- Fu, G., 2026. Toward autonomous planning and management of urban water systems. *ACS ES&T Water* 6 (5), 2703–2711. <https://doi.org/10.1021/acsestwater.5c01057>.
- Gemini, T., Google, 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities, arXiv preprint arXiv:2507.06261.
- Goldstein, Y., Perelman, G., Schuster, A., Ostfeld, A., 2025. Large language models for water distribution systems modeling and decision-making. *ArXiv* abs/2503.16191.
- Guo, D. et al., 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning.
- Hochreiter, S., Schmidhuber, J., 1997. Long short-term memory. *Neural Comput.* 9 (8), 1735–1780.
- Hosseini, S.H., et al., 2026. Making waves: a conceptual framework exploring how large language model-based multi-agent systems could reshape water engineering. *Water Res.* 291, 125157. <https://doi.org/10.1016/j.watres.2025.125157>.
- Huang, C., Zhang, L., 2025. On the Limit of Language Models as Planning Formalizers. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, pp. 4880–4904. DOI:10.18653/v1/2025.acl-long.242.
- Huang, Q., Vora, J., Liang, P., Leskovec, J., 2023. MAgentBench: Evaluating Language Agents on Machine Learning Experimentation.
- Ji, Z., et al., 2023. Survey of hallucination in natural language generation. *ACM Comput. Surv.* 55 (12). <https://doi.org/10.1145/3571730>.
- Kadiyala, L.A., Mermer, O., Samuel, D.J., Sermet, Y., Demir, I., 2024. The implementation of multimodal large language models for hydrological applications: a comparative study of GPT-4 vision, gemini, LLaVa, and multimodal-GPT. *Hydrology* 11 (9), 148.
- Kizilkaya, D., Sajja, R., Sermet, Y., Demir, I., 2025. Toward HydroLLM: a benchmark dataset for hydrology-specific knowledge assessment for large language models. *Environ. Data Sci.* 4, e31.
- Kohl, J. et al., 2024. Generative AI Toolkit – a framework for increasing the quality of LLM-based applications over their whole life cycle.
- Kojima, T., Gu, S.S., Reid, M., Matsuo, Y., Iwasawa, Y., 2022. Large language models are zero-shot reasoners. *Adv. Neural Inf. Process. Syst.* 35, 22199–22213.
- Kratzert, F., 2019. CAMELS extended maurer forcing data. *HydroShare*. <https://doi.org/10.4211/hs.17c896843cf940339c3c3496d0c1c077>.
- Kratzert, F., Klotz, D., Brenner, C., Schulz, K., Herrnegger, M., 2018. Rainfall-runoff modelling using long short-term memory (LSTM) networks. *Hydrol. Earth Syst. Sci.* 22 (11), 6005–6022.
- Kratzert, F., et al., 2019. Towards learning universal, regional, and local hydrological behaviors via machine learning applied to large-sample datasets. *Hydrol. Earth Syst. Sci.* 23 (12), 5089–5110.
- Kratzert, F., Gauch, M., Nearing, G., Klotz, D., 2022. NeuralHydrology - a python library for deep learning research in hydrology. *J. Open Source Softw.* 7, 4050.
- Lei, G., Docherty, R., Cooper, S.J., 2024. Materials science in the era of large language models: a perspective. *Digital Discovery* 3 (7), 1257–1272.
- Liu, A. et al., 2024a. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437.
- Liu, N.F., et al., 2024b. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12, 157–173. https://doi.org/10.1162/tacl_a.00638.
- Liu, Y. et al., 2024c. iTransformer: Inverted Transformers are Effective for Time Series Forecasting, International Conference on Learning Representations (ICLR).

- Luo, D., Wang, X., 2024. ModernTCN: a modern pure convolution structure for general time series analysis. *The Twelfth International Conference on Learning Representations* 1–43.
- Ma, X., Hu, H., Ren, Y., 2023. A hybrid deep learning model based on feature capture of water level influencing factors and prediction error correction for water level prediction of cascade hydropower stations under multiple time scales. *J. Hydrol.* 617, 129044.
- Ma, F., Chen, J., Dai, Z., Cai, F., Hu, Y., 2026. Autonomous inverse modeling of complex groundwater systems via a physics-integrated large language model multi-agent framework. *Water Res.* 299, 125886. <https://doi.org/10.1016/j.watres.2026.125886>.
- Maldonado, D., Cruz, E., Torres, J.A., Cruz, P.J., Benitez, S.d.P.G., 2024. Multi-agent systems: A survey about its components, framework and workflow. *IEEE Access*, 12: 80950–80975.
- Martelo, R., Ahmadiyehyazdi, K., Wang, R.-Q., 2026. Towards democratized flood risk management: an advanced AI assistant enabled by GPT-4 for enhanced interpretability and public engagement. *Environ. Model. Software* 197, 106821. <https://doi.org/10.1016/j.envsoft.2025.106821>.
- Miao, C., Hu, J., Moradkhani, H., Destouni, G., 2024. Hydrological research evolution: A large language model-based analysis of 310,000 studies published globally between 1980 and 2023. *Wiley Online Library*, pp. e2024WR038077.
- Moriassi, D.N., et al., 2007. Model evaluation guidelines for systematic quantification of accuracy in watershed simulations. *Trans. ASABE* 50, 885–900.
- Newman, A.J., et al., 2015. Development of a large-sample watershed-scale hydrometeorological data set for the contiguous USA: data set characteristics and assessment of regional variability in hydrologic model performance. *Hydrol. Earth Syst. Sci.* 19 (1), 209–223.
- Newman, A.J., et al., 2017. Benchmarking of a physically based hydrologic model. *J. Hydrometeorol.* 18 (8), 2215–2225. <https://doi.org/10.1175/JHM-D-16-0284.1>.
- Nie, Y., Nguyen, N.H., Sinthong, P., Jayaraman, K., 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers, *International Conference on Learning Representations (ICLR)*.
- OpenAI, 2025. GPT-5 Technical Report, OpenAI Technical Report.
- Ray, P.P., 2025. A survey on model context protocol: architecture, state-of-the-art, challenges and future directions. *Authorea Preprints*.
- Read, J.S., et al., 2019. Process-guided deep learning predictions of lake water temperature. *Water Resour. Res.* 55 (11), 9173–9190. <https://doi.org/10.1029/2019WR024922>.
- Ren, Y., et al., 2024. WaterGPT: training a large language model to become a hydrology expert. *Water* 16 (21), 3075.
- Shen, C., et al., 2023a. Differentiable modelling to unify machine learning and physical models for geosciences. *Nature Reviews Earth & Environment* 4 (8), 552–567.
- Shen, Y., et al., 2023b. Hugginggpt: solving AI tasks with chatgpt and its friends in hugging face. *Adv. Neural Inf. Proces. Syst.* 36, 38154–38180.
- Starace, G. et al., 2025. PaperBench: Evaluating AI's Ability to Replicate AI Research.
- Sun, M., et al., 2025. A survey on large language model-based agents for statistics and data science. *The American Statistician* 1–14. <https://doi.org/10.1080/00031305.2025.2561140>.
- Tarboton, D.G., Idaszak, R., Horsburgh, J.S., Heard, J., Maidment, D., 2014. HydroShare: Advancing Collaboration through Hydrologic Data and Model Sharing.
- Tudaji, M., Tian, F., Wu, X., 2026. HydroCraft: an efficient online platform for hydrological modeling integrated with a large language model agent. *J. Hydrol.* 677, 135801. <https://doi.org/10.1016/j.jhydrol.2026.135801>.
- Vaswani, A. et al., 2017. Attention is all you need. *Advances in neural information processing systems*, 30.
- Wang, H., et al., 2023. Scientific discovery in the age of artificial intelligence. *Nature* 620 (7972), 47–60. <https://doi.org/10.1038/s41586-023-06221-2>.
- Wang, J., Fu, G., Savic, D., 2026. EPANET-agentic: a multi-agent system for natural language-controlled simulations of water distribution networks. *Water Res.* 293, 125433. <https://doi.org/10.1016/j.watres.2026.125433>.
- Wu, H. et al., 2023. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis, *International Conference on Learning Representations (ICLR)*.
- Wu, L., Zhou, H., Ma, X., Fan, J., Zhang, F., 2019. Daily reference evapotranspiration prediction based on hybridized extreme learning machine model with bio-inspired optimization algorithms: application in contrasting climates of China. *J. Hydrol.* 577, 123960.
- Wu, H., Xu, J., Wang, J., Long, M., 2021. Autoformer: decomposition transformers with auto-correlation for long-term series forecasting. *Adv. Neural Inf. Proces. Syst.* 34, 22419–22430.
- Yang, A. et al., 2025. Qwen3 technical report.
- Yang, Y.C.E., Chiou, W., 2026. Leveraging large language models for agent-based simulation of human-water system interactions. *Water Resour. Res.* 62 (6), e2025WR042111. <https://doi.org/10.1029/2025WR042111>.
- Yang, J., et al., 2024. SWE-agent: agent-computer interfaces enable. *Autom. Softw. Eng. ArXiv*.
- Yu, C. et al., 2025. A survey on agent workflow—status and future, 2025 8th International Conference on Artificial Intelligence and Big Data (ICAIBD). *IEEE*, pp. 770–781.
- Zeng, A., Chen, M., Zhang, L., Xu, Q., 2023. Are transformers effective for time series forecasting? In: *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 11121–11128.
- Zhang, W. et al., 2025a. AgentOrchestra: A Hierarchical Multi-Agent Framework for General-Purpose Task Solving.
- Zhang, Y. et al., 2025b. MLRC-Bench: Can Language Agents Solve Machine Learning Research Challenges?.
- Zhang, Z., Valeo, C., 2026. Agentic SWMM: Auditable and Reproducible Stormwater Modelling Workflow with Agent Skills and Model Context Protocol, *AI for Engineering*, pp. 5. DOI:10.3390/aieng1010005.
- Zhang, Y., Yan, J., 2023. Crossformer: transformer utilizing cross-dimension dependency for multivariate time series forecasting. *The Eleventh International Conference on Learning representations*.
- Zhmoginov, A., Bashkirova, D., Sandler, M., 2021. Compositional Models: Multi-Task Learning and Knowledge Transfer with Modular Networks.
- Zhu, Z., et al., 2026. Large language models as calibration agents in hydrological modeling: feasibility and limitations. *Geophys. Res. Lett.* 53 (2), e2025GL120043. <https://doi.org/10.1029/2025GL120043>.